

To set up a **step-by-step procedure** to download the API manual (e.g., a PDF document) using **n8n** (or a similar platform), let's walk through the necessary steps. I'll guide you in setting up the flow to download an API manual from **Google Drive**.

## **Step-by-Step Procedure to Download API Manual in PDF**

### **Step 1: Google Drive Authentication**

Before you can download any files from **Google Drive**, you must authenticate with Google and get the necessary permissions to access your files.

#### **1. Set up Google OAuth2 Authentication:**

- Go to the **Google Cloud Console** and create **OAuth 2.0 credentials** for your app.
- Make sure the **Google Drive API** is enabled.
- Add the **OAuth 2.0 Client ID** and **Client Secret** to your **n8n credentials**.

In **n8n**:

- Go to **Credentials**.
- Add **Google OAuth2 API** and authenticate with the credentials you created in the Google Cloud Console.
- Set the correct **scope** (should include drive.readonly).

### **Step 2: Set Up the HTTP Request Node**

Once you have the authentication set up, proceed to the HTTP request node.

#### **1. Add HTTP Request Node:**

- **Method:** GET
- **URL:** `https://www.googleapis.com/drive/v3/files/{fileId}?alt=media`
  - Replace `{fileId}` with the ID of the file you want to download. This file ID can be obtained from a previous step, such as the **Google Drive Trigger** or by querying a list of files using the **Google Drive API**.
  - **alt=media** is added to download the file content (in this case, the PDF).

#### **2. Authentication:**

- Make sure that you are using **Google OAuth2 credentials** for authentication.

#### **3. Query Parameters (optional):**

- If you want to filter files or pass other parameters, you can add them in the query parameters section, but for downloading the file directly, this will not be necessary.

#### **4. Set up Headers:**

- Add **Authorization** header:
- Authorization: Bearer `{{ $credentials.oauth2.access_token }}`

This will ensure your request is authenticated and the file is accessible.

### Step 3: Add a File Download or Save Node

Once the **HTTP Request Node** has successfully retrieved the file (PDF content in this case), you need to store or save it.

1. **Add File Node** (Save the file):
  - **File Type:** PDF
  - Choose the path where the downloaded file will be stored. You can use a dynamic filename or specify a static name (e.g., api\_manual.pdf).
2. **File Name:** Set the **file name** to be saved dynamically:
  - You can dynamically create the name based on the file name you retrieved from the Google Drive response.  
Example: `{{json["name"]}}.pdf` (This will save the file with its original name from Google Drive).
3. **File Destination:** Select a directory to save the downloaded file. You can specify a directory on your server or local machine depending on your use case.

### Step 4: Execute and Test

- **Execute the workflow** and ensure the file is downloaded to the specified location.
- **Verify** that the file is in PDF format and accessible.

### Example Workflow in n8n

Here is an overview of the workflow that you need:

1. **Google Drive Trigger:** This can listen to changes in the folder where your API manual (PDF) is stored. It triggers when a new file is added.
2. **HTTP Request Node:** This retrieves the file from Google Drive using the file ID.
  - **URL:** `https://www.googleapis.com/drive/v3/files/{fileId}?alt=media`
  - Add **Authorization** header for OAuth2 authentication.
3. **File Write Node:** This saves the file to the desired location (e.g., local directory or cloud storage).

### Step 5: Download Multiple Files (Optional)

If you want to download multiple files (e.g., all API manuals or other files in a folder), follow this process:

1. **Query the Files in the Folder** using the **Google Drive API** to get the list of file IDs.
  2. For each file, use the **HTTP Request Node** to download it individually.
  3. Save each file using the **File Write Node**.
-

## Summary

1. **Authenticate with Google Drive** using OAuth2 credentials.
2. **Use an HTTP Request Node** to download the API manual PDF from Google Drive.
3. **Save the file** to your desired location using the File Write Node.

Let me know if you need further assistance with any specific part of the process, and I will guide you accordingly!

## You said:

GitHub api manual download

## ChatGPT said:

To download the **API manual** from **GitHub** using their **API**, you can follow the steps outlined below. We'll go through the process of interacting with GitHub's API to fetch a file (such as an API manual) from a repository and save it.

## Step-by-Step Guide to Download API Manual from GitHub API

### Step 1: Set Up GitHub API Access

1. **Create a GitHub Personal Access Token (PAT):**
  - Go to GitHub Personal Access Tokens.
  - Click on **Generate New Token**.
  - Give it an appropriate name, e.g., "API Manual Download".
  - Select **repo** and other necessary permissions, then click **Generate token**.
  - **Important:** Copy and save the token as it will not be shown again.
2. **Save the Token:**
  - You will use this token to authenticate requests to the GitHub API.

---

### Step 2: Make the API Request to Fetch the File

GitHub provides a REST API to interact with repositories and their contents. To download a file from a repository, you need to know the following:

- **Repository Name**
- **File Path** within the repository

1. **Set Up the API Request:**
  - **Endpoint:** GET /repos/{owner}/{repo}/contents/{path}
  - Replace {owner} with the repository owner's GitHub username, {repo} with the repository name, and {path} with the path to the file you want to download.

## Example:

- If the **repo** is openai/gpt-3 and the **file path** is /docs/api\_manual.pdf, the request would look like:
- GET https://api.github.com/repos/openai/gpt-3/contents/docs/api\_manual.pdf

## 2. Set Up Authorization:

- In the request, include the **Personal Access Token (PAT)** in the **Authorization** header.
  - Authorization: Bearer YOUR\_ACCESS\_TOKEN
  - The **Authorization** header is required for private repositories or if you want to authenticate.
- 

## Step 3: Configure the HTTP Request Node in n8n

### 1. Add HTTP Request Node:

- **Method:** GET
- **URL:**
- https://api.github.com/repos/{owner}/{repo}/contents/{path}

Replace {owner}, {repo}, and {path} with the actual values.

### 2. Authentication:

- In the **Authentication** section of the HTTP Request node, select **OAuth2** or **Basic Auth** and add your **GitHub Personal Access Token**.

### 3. Headers:

- Add **Authorization** header:
- Authorization: Bearer YOUR\_ACCESS\_TOKEN

### 4. Execute the HTTP Request:

- When you execute this request, GitHub will return the file metadata, including the **download URL**.
- 

## Step 4: Download the File

GitHub returns file contents encoded in **Base64**. You will need to decode it to save the file locally.

### 1. Response Format:

GitHub's response will look like this:

2. {
3. "name": "api\_manual.pdf",
4. "path": "docs/api\_manual.pdf",
5. "content": "Base64\_encoded\_content\_here",

6. "encoding": "base64"
  7. }
  8. **Decode Base64:**
    - Extract the **Base64 content** and decode it using a **Function Node** or a suitable n8n node.
    - The decoded content is the actual file content.
  9. **Save the File:**
    - Once the content is decoded, use a **File Node** to save the file to a specified location.
      - **File Name:** api\_manual.pdf
      - **File Content:** The decoded content from the previous step.
- 

## Step 5: Complete Workflow

Here's how your **n8n workflow** will look:

1. **HTTP Request Node** (GET file metadata from GitHub)
  2. **Function Node** (Decode Base64 content)
  3. **File Node** (Save the decoded content as a PDF file)
- 

## Example of the Full Workflow in n8n

1. **HTTP Request Node:**
  - **Method:** GET
  - **URL:** [https://api.github.com/repos/openai/gpt-3/contents/docs/api\\_manual.pdf](https://api.github.com/repos/openai/gpt-3/contents/docs/api_manual.pdf)
  - **Headers:**
    - Authorization: Bearer YOUR\_ACCESS\_TOKEN
2. **Function Node:**
  - Use the **Function Node** to decode the Base64 content returned by GitHub.
  - `const base64Content = $json["content"];`
  - `const decodedContent = Buffer.from(base64Content, 'base64');`
  - `return [{ json: { fileContent: decodedContent } }];`
3. **File Node:**
  - **Action:** Write to a file
  - **File Name:** api\_manual.pdf

- **File Content:** `{{ $json["fileContent"] }}`
- 

### Step 6: Test the Workflow

1. **Run the Workflow** and check if the **PDF file** is correctly downloaded and saved.
  2. **Check the directory** where the file is saved, and ensure it's accessible.
- 

### Conclusion

With these steps, you should be able to:

1. Authenticate with GitHub using a **Personal Access Token (PAT)**.
2. Fetch the file from GitHub using the **GET** API request.
3. Decode the **Base64 content** and save the file locally.

Let me know if you need more help with specific parts of the setup!

#### You said:

No such download of entire file

#### ChatGPT said:

I understand now that you're looking for a more specific **step-by-step guide** to download the **entire API manual file** (e.g., a PDF file) from **GitHub** using its **API**, and how to do this for **all files in a repository**.

Let me break down the exact steps you need to **download the entire file** from GitHub:

#### Goal:

We want to download an entire file (such as an **API Manual** stored in a GitHub repository) using the **GitHub API**.

---

### Step-by-Step Guide to Download the Entire API Manual File from GitHub

---

#### Step 1: Set Up Personal Access Token (PAT) in GitHub

To interact with the GitHub API, you'll need to authenticate. The simplest way is by using **Personal Access Tokens (PAT)**.

1. **Go to GitHub Settings:** GitHub Tokens Page
2. **Create a new token:**
  - Give it an appropriate name (e.g., "API Manual Download").
  - Select the required scopes. The minimal scope for reading repository files is repo.

3. **Save the Token:** Copy the token, as you'll use it to authenticate the requests.
- 

## Step 2: Set Up HTTP Request in n8n

Now, you need to use the **GitHub API** to download the file. This is done through an **HTTP Request** node in **n8n**.

1. **Add HTTP Request Node:**

- **Method:** GET
- **URL:**
  - `https://api.github.com/repos/{owner}/{repo}/contents/{path}`
    - **{owner}**: The GitHub username (e.g., openai).
    - **{repo}**: The repository name (e.g., gpt-3).
    - **{path}**: The file path in the repository (e.g., docs/api\_manual.pdf).

2. **Authentication:**

- Under **Authorization**, choose **Bearer Token**.
- Paste your **GitHub Personal Access Token** in the token field.

3. **Configure Headers:**

Add an **Authorization** header:

4. Authorization: Bearer YOUR\_ACCESS\_TOKEN
- 

## Step 3: Process the File Data

GitHub will respond with the file's metadata, including the **Base64-encoded content** of the file. We will decode the content and store the file.

1. **Response Format from GitHub:**

GitHub's response will look something like this:

2. {
3. "name": "api\_manual.pdf",
4. "path": "docs/api\_manual.pdf",
5. "content": "Base64\_encoded\_content\_here",
6. "encoding": "base64"
7. }

8. **Decode Base64:**

In the **Function Node**, decode the Base64 content to get the file data.

#### 9. **Example of Function Node Code:**

In the **Function Node**, add the following code to decode the Base64 content:

10. `const base64Content = $json["content"]; // Get the Base64 content`
  11. `const decodedContent = Buffer.from(base64Content, 'base64'); // Decode the Base64 content`
  12. `return [{ json: { fileContent: decodedContent } }];`
  13. **Save the File:**  
Use the **File Write Node** to save the decoded content to a specific location.
- 

### Step 4: Save the File to Your Local Machine

1. **Add a File Write Node:**
    - **Action:** Write to a file.
    - **File Name:** Set a static name like `api_manual.pdf` or use dynamic content from the GitHub response (`{{ $json["name"] }}`).
    - **File Content:** Set it as `{{ $json["fileContent"] }}` (from the Function Node's output).
  2. **Save Location:** Choose where you want the file to be saved, either locally or in a cloud storage service.
- 

### Step 5: Test the Workflow

1. **Execute the workflow** in n8n.
  2. Verify the file **API manual (PDF)** has been correctly downloaded to the specified location.
- 

### Example: Complete n8n Workflow

1. **HTTP Request Node:**
    - Fetch the **API manual** from GitHub using the GET method and the correct URL and parameters.
  2. **Function Node:**
    - Decode the Base64 content from GitHub.
  3. **File Write Node:**
    - Save the decoded content to a file.
- 

### Final Thoughts

1. Ensure that the **correct file path** is provided in the API request to GitHub.



2. The **Base64 content** from GitHub needs to be decoded and saved to the file system properly.
3. Handle **pagination** if you plan to download multiple files, especially if they are spread across multiple folders or if there are too many files.

Let me know if you'd like to walk through each step in **n8n** or need further clarification.

## About the REST API

Get oriented to the REST API documentation.

### In this article

- 

[Getting started with the REST API](#)

- 

[Further reading](#)

You can use GitHub's API to build scripts and applications that automate processes, integrate with GitHub, and extend GitHub. For example, you could use the API to triage issues, build an analytics dashboard, or manage releases.

Each REST API endpoint is documented individually, and the endpoints are categorized by the resource that they primarily affect. For example, you can find endpoints relating to issues in [REST API endpoints for issues](#).

### [Getting started with the REST API](#)

**If you are new to REST APIs**, you may find it helpful to refer to the Quickstart or Getting Started guide for an introduction. For more information, see:

- [Quickstart for GitHub REST API](#)
- [Getting started with the REST API](#)

**If you are familiar with REST APIs** but new to GitHub's REST API, you may find it helpful to refer to the authentication documentation. For more information, see:

- [Authenticating to the REST API](#)

**If you are building scripts or applications** that use the REST API, you may find some of the following guides helpful. For examples of scripting with the REST API, see:

- [Scripting with the REST API and JavaScript](#)
- [Scripting with the REST API and Ruby](#)
- [Building a GitHub App that responds to webhook events](#)
- [Building a CLI with a GitHub App](#)

- [Automatically redelivering failed deliveries for a repository webhook](#)

For a list of libraries to facilitate scripting with the REST API, see [Libraries for the REST API](#).

If you are building scripts or applications that use the REST API, you might also be interested in using webhooks to get notified about events or a GitHub App to access resources on behalf of a user or in an organization. For more information, see [About webhooks](#) and [Deciding when to build a GitHub App](#).

### **[Further reading](#)**

- [Comparing GitHub's REST API and GraphQL API](#)
- [Best practices for using the REST API](#)
- [Keeping your API credentials secure](#)
- [Troubleshooting the REST API](#)

## **Comparing GitHub's REST API and GraphQL API**

Learn about GitHub's APIs to extend and customize your GitHub experience.

### **In this article**

- 

[About GitHub's APIs](#)

- 

[Choosing the GraphQL API](#)

- 

[Choosing the REST API](#)

### **[About GitHub's APIs](#)**

GitHub provides two APIs: a REST API and a GraphQL API. You can interact with both APIs using GitHub CLI, curl, the official Octokit libraries, and third party libraries. Occasionally, a feature may be supported on one API but not the other.

You should use the API that best aligns with your needs and that you are most comfortable using. You don't need to exclusively use one API over the other. Node IDs let you move between the REST API and GraphQL API. For more information, see [Using global node IDs](#).

This article discusses the benefits of each API. For more information about the GraphQL API, see [About the GraphQL API](#). For more information about the REST API, see [About the REST API](#).

[Choosing the GraphQL API](#)

The GraphQL API returns exactly the data that you request. GraphQL also returns the data in a pre-known structure based on your request. In contrast, the REST API returns more data than you requested and returns it in a pre-determined structure. You can also accomplish the equivalent of multiple REST API request in a single GraphQL request. The ability to make fewer requests and fetch less data makes GraphQL appealing to developers of mobile applications.

For example, to get the GitHub login of ten of your followers, and the login of ten followers of each of your followers, you can send a single request like:

```
{
  viewer {
    followers(first: 10) {
      nodes {
        login
        followers(first: 10) {
          nodes {
            login
          }
        }
      }
    }
  }
}
```

The response will be a JSON object that follows the structure of your request.

In contrast, to get this same information from the REST API, you would need to first make a request to GET /user/followers. The API would return the login of each follower, along with other data about the followers that you don't need. Then, for each follower, you would need to make a request to GET /users/{username}/followers. In total, you would need to make 11 requests to get the same information that you could get from a single GraphQL request, and you would receive excess data.

### [Choosing the REST API](#)

Because REST APIs have been around for longer than GraphQL APIs, some developers are more comfortable with the REST API. Since REST APIs use standard HTTP verbs and concepts, many developers are already familiar with the basic concepts to use the REST API.

For example, to create an issue in the octocat/Spoon-Knife repository, you would need to send a request to POST /repos/octocat/Spoon-Knife/issues with a JSON request body:

```
{
  "title": "Bug with feature X",
```

```
"body": "If you do A, then B happens"
}
```

In contrast, to make an issue using the GraphQL API, you would need to get the node ID of the octocat/Spoon-Knife repository and then send a request like:

```
mutation {
  createIssue(
    input: {
      repositoryId: "MDEwOlJlcG9zaXRvcnkzMzAwMTky"
      title: "Bug with feature X"
      body: "If you do A, then B happens"
    }
  ) {
    issue {
      number
      url
    }
  }
}
```

## API Versions

Learn how to specify which REST API version to use whenever you make a request to the REST API.

### In this article

- 

[About API versioning](#)

- 

[Specifying an API version](#)

- 

[Upgrading to a new API version](#)

- 

[Supported API versions](#)

## [About API versioning](#)

The GitHub REST API is versioned. The API version name is based on the date when the API version was released. For example, the API version 2022-11-28 was released on Mon, 28 Nov 2022.

Any breaking changes will be released in a new API version. Breaking changes are changes that can potentially break an integration. Breaking changes include:

- Removing an entire operation
- Removing or renaming a parameter
- Removing or renaming a response field
- Adding a new required parameter
- Making a previously optional parameter required
- Changing the type of a parameter or response field
- Removing enum values
- Adding a new validation rule to an existing parameter
- Changing authentication or authorization requirements

Any additive (non-breaking) changes will be available in all supported API versions. Additive changes are changes that should not break an integration. Additive changes include:

- Adding an operation
- Adding an optional parameter
- Adding an optional request header
- Adding a response field
- Adding a response header
- Adding enum values

When a new REST API version is released, the previous API version will be supported for at least 24 more months following the release of the new API version.

## [Specifying an API version](#)

You should use the X-GitHub-API-Version header to specify an API version. For example:

```
curl --header "X-GitHub-API-Version:2022-11-28" https://api.github.com/zen
```

Requests without the X-GitHub-API-Version header will default to use the 2022-11-28 version.

If you specify an API version that is no longer supported, you will receive a 400 error.

## [Upgrading to a new API version](#)

Before upgrading to a new REST API version, you should read the changelog of breaking changes for the new API version to understand what breaking changes are included and to learn more about how to upgrade to that specific API version. For more information, see [Breaking changes](#).

When you update your integration to specify the new API version in the X-GitHub-API-Version header, you'll also need to make any changes required for your integration to work with the new API version.

Once your integration is updated, test your integration to verify that it works with the new API version.

### [Supported API versions](#)

The following REST API versions are currently supported:

2022-11-28

You can also make an API request to get all of the supported API versions. For more information, see [REST API endpoints for meta data](#).

### **Breaking changes**

Learn about breaking changes that were introduced in each REST API version.

#### **In this article**

- 

[About breaking changes in the REST API](#)

- 

[Upgrading to a new API version](#)

- 

[Breaking changes for 2022-11-28](#)

### [About breaking changes in the REST API](#)

The GitHub REST API is versioned. The API version name is based on the date when the API version was released. For example, the API version 2022-11-28 was released on Mon, 28 Nov 2022.

Any breaking changes will be released in a new API version. Breaking changes are changes that can potentially break an integration. Breaking changes include:

- Removing an entire operation
- Removing or renaming a parameter
- Removing or renaming a response field
- Adding a new required parameter

- Making a previously optional parameter required
- Changing the type of a parameter or response field
- Removing enum values
- Adding a new validation rule to an existing parameter
- Changing authentication or authorization requirements

Any additive (non-breaking) changes will be available in all supported API versions. Additive changes are changes that should not break an integration. Additive changes include:

- Adding an operation
- Adding an optional parameter
- Adding an optional request header
- Adding a response field
- Adding a response header
- Adding enum values

When a new REST API version is released, the previous API version will be supported for at least 24 more months following the release of the new API version.

For more information about API versions, see [API Versions](#).

### [Upgrading to a new API version](#)

Before upgrading to a new REST API version, you should read the section on this page that corresponds to the new API version to understand what breaking changes are included and to learn more about how to upgrade to that API version.

When you update your integration to specify the new API version in the X-GitHub-API-Version header, you'll also need to make any changes required for your integration to work with the new API version.

Once your integration is updated, test your integration to verify that it works with the new API version.

### [Breaking changes for 2022-11-28](#)

Version 2022-11-28 is the first version of the GitHub REST API after date-based versioning was introduced. This version does not include any breaking changes.

### **About the OpenAPI description for the REST API**

The GitHub REST API is fully described in an OpenAPI compliant document.

### **In this article**

- 

[About OpenAPI](#)

- 

[About GitHub's OpenAPI description](#)

- 

[Using the GitHub OpenAPI description](#)

## [About OpenAPI](#)

OpenAPI is a specification for describing REST API interfaces. It describes the API without requiring access to the source code or additional documentation. The specification is both human and machine readable. For more information, see [the OpenAPI specification documentation](#).

## [About GitHub's OpenAPI description](#)

GitHub's OpenAPI description of the REST API is publicly available. You can find the description in the open source [github/rest-api-description](#) repository.

GitHub provides both 3.0 and 3.1 OpenAPI descriptions.

For each description, there is a version for each product: GitHub Free/GitHub Pro/GitHub Team (api.github.com), GitHub Enterprise Cloud (ghec), and each version of GitHub Enterprise Server (ghes-X.X).

For each product, if date-based versioning is supported, there is also a description for each date-based version. For more information, see [API Versions](#).

Each description is available in a bundled or in a dereferenced format. The bundled format uses \$ref to refer to OpenAPI components that are shared between endpoints. The dereferenced format includes the fully expanded description.

## [Using the GitHub OpenAPI description](#)

Because the OpenAPI description is machine readable, you can use it to do things like:

- Generate libraries to facilitate using the REST API
- Validate and test an integration that uses the REST API
- Explore and interact with the REST API using third-party tools, such as Insomnia or Postman

For example, GitHub uses the OpenAPI description to generate the Octokit SDKs. GitHub also uses the OpenAPI description to generate the REST API reference documentation for each endpoint.

## [Using the GitHub OpenAPI description](#)

Because the OpenAPI description is machine readable, you can use it to do things like:

- Generate libraries to facilitate using the REST API
- Validate and test an integration that uses the REST API
- Explore and interact with the REST API using third-party tools, such as Insomnia or Postman



For example, GitHub uses the OpenAPI description to generate the Octokit SDKs. GitHub also uses the OpenAPI description to generate the REST API reference documentation for each endpoint.

## Getting started with the REST API

Learn how to use the GitHub REST API.

### Tool navigation

- [GitHub CLI](#)
- [curl](#)
- [JavaScript](#)

### In this article

- 

#### [Introduction](#)

- 

#### [About requests to the REST API](#)

- 

#### [Making a request](#)

- 

#### [Using the response](#)

- 

#### [Next steps](#)

## [Introduction](#)

This article describes how to use the GitHub REST API with GitHub CLI, curl, or JavaScript. For a quickstart guide, see [Quickstart for GitHub REST API](#).

## [About requests to the REST API](#)

This section describes the elements that make up an API request:

- [HTTP method](#)
- [Path](#)
- [Headers](#)

- [Media types](#)
- [Authentication](#)
- [Parameters](#)

Every request to the REST API includes an HTTP method and a path. Depending on the REST API endpoint, you might also need to specify request headers, authentication information, query parameters, or body parameters.

The REST API reference documentation describes the HTTP method, path, and parameters for every endpoint. It also displays example requests and responses for each endpoint. For more information, see the [REST reference documentation](#).

### [HTTP method](#)

The HTTP method of an endpoint defines the type of action it performs on a given resource. Some common HTTP methods are GET, POST, DELETE, and PATCH. The REST API reference documentation provides the HTTP method for every endpoint.

For example, the HTTP method for the ["List repository issues" endpoint](#) is GET."

Where possible, the GitHub REST API strives to use an appropriate HTTP method for each action.

- GET: Used for retrieving resources.
- POST: Used for creating resources.
- PATCH: Used for updating properties of resources.
- PUT: Used for replacing resources or collections of resources.
- DELETE: Used for deleting resources.

### [Path](#)

Each endpoint has a path. The REST API reference documentation gives the path for every endpoint. For example, the path for the ["List repository issues" endpoint](#) is `/repos/{owner}/{repo}/issues`.

The curly brackets `{}` in a path denote path parameters that you need to specify. Path parameters modify the endpoint path and are required in your request. For example, the path parameters for the ["List repository issues" endpoint](#) are `{owner}` and `{repo}`. To use this path in your API request, replace `{repo}` with the name of the repository where you would like to request a list of issues, and replace `{owner}` with the name of the account that owns the repository.

### [Headers](#)

Headers provide extra information about the request and the desired response. Following are some examples of headers that you can use in your requests to the GitHub REST API. For an example of a request that uses headers, see [Making a request](#).

### [Accept](#)

Most GitHub REST API endpoints specify that you should pass an Accept header with a value of `application/vnd.github+json`. The value of the Accept header is a media type. For more information about media types, see [Media types](#).

## [X-GitHub-API-Version](#)

You should use this header to specify a version of the REST API to use for your request. For more information, see [API Versions](#).

## [User-Agent](#)

All API requests must include a valid User-Agent header. The User-Agent header identifies the user or application that is making the request.

By default, GitHub CLI sends a valid User-Agent header. However, GitHub recommends using your GitHub username, or the name of your application, for the User-Agent header value. This allows GitHub to contact you if there are problems.

The following is an example User-Agent for an app named Awesome-Octocat-App:

User-Agent: Awesome-Octocat-App

Requests with no User-Agent header will be rejected. If you provide an invalid User-Agent header, you will receive a 403 Forbidden response.

## [Media types](#)

You can specify one or more media types by adding them to the Accept header of your request. For more information about the Accept header, see [Accept](#).

Media types specify the format of the data you want to consume from the API. Media types are specific to resources, allowing them to change independently and support formats that other resources don't. The documentation for each GitHub REST API endpoint will describe the media types that it supports. For more information, see the [GitHub REST API documentation](#).

The most common media types supported by the GitHub REST API are application/vnd.github+json and application/json.

There are custom media types that you can use with some endpoints. For example, the REST API to manage [commits](#) and [pull requests](#) support the media types diff, patch, and sha. The media types full, raw, text, or html are used by some other endpoints.

All custom media types for GitHub look like this: application/vnd.github.PARAM+json, where PARAM is the name of the media type. For example, to specify the raw media type, you would use application/vnd.github.raw+json.

For an example of a request that uses media types, see [Making a request](#).

## [Authentication](#)

Many endpoints require authentication or return additional information if you are authenticated. Additionally, you can make more requests per hour when you are authenticated.

Although some REST API endpoints are accessible without authentication, GitHub CLI requires you to authenticate before you can use the api subcommand to make an API request. Use the auth login subcommand to authenticate to GitHub. For more information, see [Making a request](#).

## [Parameters](#)

Many API methods require or allow you to send additional information in parameters in your request. There are a few different types of parameters: Path parameters, body parameters, and query parameters.

### [Path parameters](#)

Path parameters modify the endpoint path. These parameters are required in your request. For more information, see [Path](#).

### [Body parameters](#)

Body parameters allow you to pass additional data to the API. These parameters can be optional or required, depending on the endpoint. For example, a body parameter may allow you to specify an issue title when creating a new issue, or specify certain settings when enabling or disabling a feature. The documentation for each GitHub REST API endpoint will describe the body parameters that it supports. For more information, see the [GitHub REST API documentation](#).

For example, the ["Create an issue" endpoint](#) requires that you specify a title for the new issue in your request. It also allows you to optionally specify other information, such as text to put in the issue body, users to assign to the new issue, or labels to apply to the new issue. For an example of a request that uses body parameters, see [Making a request](#).

You must authenticate your request to pass body parameters. For more information, see [Authenticating](#).

### [Query parameters](#)

Query parameters allow you to control what data is returned for a request. These parameters are usually optional. The documentation for each GitHub REST API endpoint will describe any query parameters that it supports. For more information, see the [GitHub REST API documentation](#).

For example, the ["List public events" endpoint](#) returns thirty issues by default. You can use the `per_page` query parameter to return two issues instead of 30. You can use the `page` query parameter to fetch only the first page of results. For an example of a request that uses query parameters, see [Making a request](#).

### [Making a request](#)

This section demonstrates how to make an authenticated request to the GitHub REST API using GitHub CLI.

#### [1. Setup](#)

Install GitHub CLI on macOS, Windows, or Linux. For more information, see [Installation](#) in the GitHub CLI repository.

#### [2. Authenticate](#)

1. To authenticate to GitHub, run the following command from your terminal.
2. `gh auth login`

You can use the `--scopes` option to specify what scopes you want. If you want to authenticate with a token that you created, you can use the `--with-token` option. For more information, see the [GitHub CLI auth login documentation](#).

3. Select where you want to authenticate to:
  - If you access GitHub at GitHub.com, select **GitHub.com**.
  - If you access GitHub at a different domain, select **Other**, then enter your hostname (for example: octocorp.ghe.com).
4. Follow the rest of the on-screen prompts.

GitHub CLI automatically stores your Git credentials for you when you choose HTTPS as your preferred protocol for Git operations and answer "yes" to the prompt asking if you would like to authenticate to Git with your GitHub credentials. This can be useful as it allows you to use Git commands like `git push` and `git pull` without needing to set up a separate credential manager or use SSH.

### 3. Choose an endpoint for your request

1. Choose an endpoint to make a request to. You can explore GitHub's [REST API documentation](#) to discover endpoints that you can use to interact with GitHub.
2. Identify the HTTP method and path of the endpoint. You will send these with your request. For more information, see [HTTP method](#) and [Path](#).

For example, the ["Create an issue" endpoint](#) uses the HTTP method POST and the path `/repos/{owner}/{repo}/issues`.

3. Identify any required path parameters. Required path parameters appear in curly brackets `{}` in the path of the endpoint. Replace each parameter placeholder with the desired value. For more information, see [Path](#).

For example, the ["Create an issue" endpoint](#) uses the path `/repos/{owner}/{repo}/issues`, and the path parameters are `{owner}` and `{repo}`. To use this path in your API request, replace `{repo}` with the name of the repository where you would like to create a new issue, and replace `{owner}` with the name of the account that owns the repository.

### 4. Make a request with GitHub CLI

Use the GitHub CLI `api` subcommand to make your API request. For more information, see the [GitHub CLI api documentation](#).

In your request, specify the following options and values:

- **--method** followed by the HTTP method and the path of the endpoint. For more information, see [HTTP method](#) and [Path](#).
- **--header:**
  - **Accept:** Pass the media type in an Accept header. To pass multiple media types in an Accept header, separate the media types with a comma: `Accept: application/vnd.github+json,application/vnd.github.diff`. For more information, see [Accept](#) and [Media types](#).
  - **X-GitHub-API-Version:** Pass the API version in a X-GitHub-API-Version header. For more information, see [X-GitHub-API-Version](#).

- **-f** or **-F** followed by any body parameters or query parameters in key=value format. Use the **-F** option to pass a parameter that is a number, Boolean, or null. Use the **-f** option to pass string parameters.

Some endpoints use query parameters that are arrays. To send an array in the query string, use the query parameter once per array item, and append `[]` after the query parameter name. For example, to provide an array of two repository IDs, use `-f repository_ids[]=REPOSITORY_A_ID -f repository_ids[]=REPOSITORY_B_ID`.

If you do not need to specify any body parameters or query parameters in your request, omit this option. For more information, see [Body parameters](#) and [Query parameters](#). For examples, see [Example request using body parameters](#) and [Example request using query parameters](#).

### [Example request](#)

The following example request uses the ["Get Octocat" endpoint](#) to return the octocat as ASCII art.

Shell

```
gh api --method GET /octocat \
--header 'Accept: application/vnd.github+json' \
--header "X-GitHub-API-Version: 2022-11-28"
```

### [Example request using query parameters](#)

The ["List public events" endpoint](#) returns thirty issues by default. The following example uses the `per_page` query parameter to return two issues instead of 30, and the `page` query parameter to fetch only the first page of results.

Shell

```
gh api --method GET /events -F per_page=2 -F page=1
--header 'Accept: application/vnd.github+json' \
```

### [Example request using body parameters](#)

The following example uses the ["Create an issue" endpoint](#) to create a new issue in the octocat/Spoon-Knife repository. In the response, find the `html_url` of your issue, and navigate to your issue in the browser.

Shell

```
gh api --method POST /repos/octocat/Spoon-Knife/issues \
--header "Accept: application/vnd.github+json" \
--header "X-GitHub-API-Version: 2022-11-28" \
-f title='Created with the REST API' \
-f body='This is a test issue created by the REST API' \
```

### [Using the response](#)

After you make a request, the API will return the response status code, response headers, and potentially a response body.

### About the response code and headers

Every request will return an HTTP status code that indicates the success of the response. For more information about response codes, see [the MDN HTTP response status code documentation](#).

Additionally, the response will include headers that give more details about the response. Headers that start with X- or x- are custom to GitHub. For example, the x-ratelimit-remaining and x-ratelimit-reset headers tell you how many requests you can make in a time period.

To view the status code and headers, use the --include or --i option when you send your request.

For example, this request gets a list of issues in the octocat/Spoon-Knife repository:

```
gh api \
--header 'Accept: application/vnd.github+json' \
--method GET /repos/octocat/Spoon-Knife/issues \
-F per_page=2 --include
```

And it returns a response code and headers that look something like this:

HTTP/2.0 200 OK

Access-Control-Allow-Origin: \*

Access-Control-Expose-Headers: ETag, Link, Location, Retry-After, X-RateLimit-Limit, X-RateLimit-Remaining, X-RateLimit-Used, X-RateLimit-Resource, X-RateLimit-Reset, X-OAuth-Scopes, X-Accepted-OAuth-Scopes, X-Poll-Interval, X-GitHub-Media-Type, X-GitHub-SSO, X-GitHub-Request-Id, Deprecation, Sunset

Cache-Control: private, max-age=60, s-maxage=60

Content-Security-Policy: default-src 'none'

Content-Type: application/json; charset=utf-8

Date: Thu, 04 Aug 2022 19:56:41 GMT

Etag: W/"a63dfbcfdb73621e9d2e89551edcf9856731ced534bd7f1e114a5da1f5f73418"

Link: <https://api.github.com/repositories/1300192/issues?per\_page=1&page=2>; rel="next",  
<https://api.github.com/repositories/1300192/issues?per\_page=1&page=14817>; rel="last"

Referrer-Policy: origin-when-cross-origin, strict-origin-when-cross-origin

Server: GitHub.com

Strict-Transport-Security: max-age=31536000; includeSubdomains; preload

Vary: Accept, Authorization, Cookie, Accept-Encoding, Accept, X-Requested-With

X-Accepted-Oauth-Scopes: repo

X-Content-Type-Options: nosniff

X-Frame-Options: deny  
X-Github-API-Version-Selected: 2022-08-09  
X-Github-Media-Type: github.v3; format=json  
X-Github-Request-Id: 1C73:26D4:E2E500:1EF78F4:62EC2479  
X-Oauth-Client-Id: 178c6fc778ccc68e1d6a  
X-Oauth-Scopes: gist, read:org, repo, workflow  
X-Ratelimit-Limit: 15000  
X-Ratelimit-Remaining: 14996  
X-Ratelimit-Reset: 1659645499  
X-Ratelimit-Resource: core  
X-Ratelimit-Used: 4  
X-Xss-Protection: 0

In this example, the response code is 200, which indicates a successful request.

### [About the response body](#)

Many endpoints will return a response body. Unless otherwise specified, the response body is in JSON format. Blank fields are included as null instead of being omitted. All timestamps return in UTC time, ISO 8601 format: YYYY-MM-DDTHH:MM:SSZ.

Unlike the GraphQL API where you specify what information you want, the REST API typically returns more information than you need. If desired, you can parse the response to pull out specific pieces of information.

For example, you can use `>` to redirect the response to a file. In the following example, replace REPO-OWNER with the name of the account that owns the repository, and REPO-NAME with the name of the repository.

```
Shell
gh api \
--header 'Accept: application/vnd.github+json' \
--method GET /repos/REPO-OWNER/REPO-NAME/issues \
-F per_page=2 > data.json
```

Then you can use `jq` to get the title and author ID of each issue:

```
Shell
jq '.[] | {title: .title, authorID: .user.id}' data.json
```

The previous two commands return something like:

```
{
```



```
"title": "Update index.html",  
"authorID": 10701255  
}  
  
{  
  "title": "Edit index file",  
  "authorID": 53709285  
}
```

For more information about jq, see [the jq documentation](#).

### [Detailed versus summary representations](#)

A response can include all attributes for a resource or only a subset of attributes, depending on whether you fetch an individual resource or a list of resources.

- When you fetch an *individual resource*, like a specific repository, the response will typically include all attributes for that resource. This is the "detailed" representation of the resource.
- When you fetch a *list of resources*, like a list of multiple repositories, the response will only include a subset of the attributes for each resource. This is the "summary" representation of the resource.

Note that authorization sometimes influences the amount of detail included in a representation.

The reason for this is because some attributes are computationally expensive for the API to provide, so GitHub excludes those attributes from the summary representation. To obtain those attributes, you can fetch the detailed representation.

The documentation provides an example response for each API method. The example response illustrates all attributes that are returned by that method.

### [Hypermedia](#)

All resources may have one or more `*_url` properties linking to other resources. These are meant to provide explicit URLs so that proper API clients don't need to construct URLs on their own. It is highly recommended that API clients use these. Doing so will make future upgrades of the API easier for developers. All URLs are expected to be proper [RFC 6570](#) URI templates.

You can then expand these templates using something like the [uri\\_template](#) gem:

```
>> tmpl = URI::Template.new('/notifications{?since,all,participating}')  
  
>> tmpl.expand  
=> "/notifications"  
  
  
>> tmpl.expand all: 1  
=> "/notifications?all=1"
```

>> tmpl.expand all: 1, participating: 1

=> "/notifications?all=1&participating=1"

### [Next steps](#)

This article demonstrated how to list and create issues in a repository. For more practice, try to comment on an issue, edit the title of an issue, or close an issue. For more information, see the ["Create an issue comment" endpoint](#) and the ["Update an issue" endpoint](#).

For more information about other endpoints that you can use, see the [REST reference documentation](#).

### **Rate limits for the REST API**

Learn about REST API rate limits, how to avoid exceeding them, and what to do if you do exceed them.

#### **In this article**

- 

[About primary rate limits](#)

- 

[About secondary rate limits](#)

- 

[Checking the status of your rate limit](#)

- 

[Exceeding the rate limit](#)

- 

[Staying under the rate limit](#)

- 

[Getting a higher rate limit](#)

[About primary rate limits](#)

GitHub limits the number of REST API requests that you can make within a specific amount of time. This limit helps prevent abuse and denial-of-service attacks, and ensures that the API remains available for all users.

Some endpoints, like the search endpoints, have more restrictive limits. For more information about these endpoints, see [REST API endpoints for rate limits](#). The GraphQL API also has a separate primary rate limit. See [Rate limits and node limits for the GraphQL API](#).

In general, you can calculate your primary rate limit for the REST API based on your method of authentication, as described below.

#### **Primary rate limit for unauthenticated users**

You can make unauthenticated requests if you are only fetching public data. Unauthenticated requests are associated with the originating IP address, not with the user or application that made the request.

The primary rate limit for unauthenticated requests is 60 requests per hour.

#### **Primary rate limit for authenticated users**

You can use a personal access token to make API requests. Additionally, you can authorize a GitHub App or OAuth app, which can then make API requests on your behalf.

All of these requests count towards your personal rate limit of 5,000 requests per hour. Requests made on your behalf by a GitHub App that is owned by a GitHub Enterprise Cloud organization have a higher rate limit of 15,000 requests per hour. Similarly, requests made on your behalf by a OAuth app that is owned or approved by a GitHub Enterprise Cloud organization have a higher rate limit of 15,000 requests per hour if you are a member of the GitHub Enterprise Cloud organization.

#### **Primary rate limit for Git LFS access**

API requests are required when you upload or download Git LFS content. These count towards a separate rate limiting bucket with a limit of 300 requests per minute for unauthenticated requests and 3,000 requests per minute for authenticated requests.

Git LFS uses a batch API which processes 100 Git LFS objects per API request by default. That means unauthenticated users can download 30,000 Git LFS objects per minute and authenticated users can upload/download 300,000 Git LFS objects per minute.

#### **Primary rate limit for GitHub App installations**

GitHub Apps authenticating with an installation access token use the installation's minimum rate limit of 5,000 requests per hour. If the installation is on a GitHub Enterprise Cloud organization, the installation has a rate limit of 15,000 requests per hour.

For installations that are not on a GitHub Enterprise Cloud organization, the rate limit for the installation will scale with the number of users and repositories. Installations that have more than 20 repositories receive another 50 requests per hour for each repository. Installations that are on an organization that have more than 20 users receive another 50 requests per hour for each user. The rate limit cannot increase beyond 12,500 requests per hour.

Primary rate limits for GitHub App user access tokens (as opposed to installation access tokens) are dictated by the primary rate limits for the authenticated user. This rate limit is combined with any

requests that another GitHub App or OAuth app makes on that user's behalf and any requests that the user makes with a personal access token. For more information, see [Rate limits for the REST API](#).

### [Primary rate limit for OAuth apps](#)

Primary rate limits for OAuth access tokens generated by a OAuth app are dictated by the primary rate limits for authenticated users. This rate limit is combined with any requests that another GitHub App or OAuth app makes on that user's behalf and any requests that the user makes with a personal access token. See [Primary rate limit for authenticated users](#).

OAuth apps can also use their client ID and client secret to fetch public data. For example:

```
curl -u YOUR_CLIENT_ID:YOUR_CLIENT_SECRET -I https://api.github.com/meta
```

For these requests, the rate limit is 5,000 requests per hour per OAuth app. If the app is owned by a GitHub Enterprise Cloud organization, the rate limit is 15,000 requests per hour.

### Note

Never include your app's client secret in client-side code or in code that runs on a user device. The client secret can be used to generate OAuth access tokens for users who have authorized your app, so you should always keep the client secret secure.

### [Primary rate limit for GITHUB\\_TOKEN in GitHub Actions](#)

You can use the built-in GITHUB\_TOKEN to authenticate requests in GitHub Actions workflows. See [Use GITHUB\\_TOKEN for authentication in workflows](#).

The rate limit for GITHUB\_TOKEN is 1,000 requests per hour per repository. For requests to resources that belong to a GitHub Enterprise Cloud account, the limit is 15,000 requests per hour per repository.

### [About secondary rate limits](#)

In addition to primary rate limits, GitHub enforces secondary rate limits in order to prevent abuse and keep the API available for all users.

You may encounter a secondary rate limit if you:

- *Make too many concurrent requests.* No more than 100 concurrent requests are allowed. This limit is shared across the REST API and GraphQL API.
- *Make too many requests to a single endpoint per minute.* No more than 900 points per minute are allowed for REST API endpoints, and no more than 2,000 points per minute are allowed for the GraphQL API endpoint. For more information about points, see [Calculating points for the secondary rate limit](#).
- *Make too many requests per minute.* No more than 90 seconds of CPU time per 60 seconds of real time is allowed. No more than 60 seconds of this CPU time may be for the GraphQL API. You can roughly estimate the CPU time by measuring the total response time for your API requests.
- *Make too many requests that consume excessive compute resources in a short period of time.*
- *Create too much content on GitHub in a short amount of time.* In general, no more than 80 content-generating requests per minute and no more than 500 content-generating requests

per hour are allowed. Some endpoints have lower content creation limits. Content creation limits include actions taken on the GitHub web interface as well as via the REST API and GraphQL API.

These secondary rate limits are subject to change without notice. You may also encounter a secondary rate limit for undisclosed reasons.

### [Calculating points for the secondary rate limit](#)

Some secondary rate limits are determined by the point values of requests. For GraphQL requests, these point values are separate from the point value calculations for the primary rate limit.

| Request                                            | Points |
|----------------------------------------------------|--------|
| GraphQL requests without mutations                 | 1      |
| GraphQL requests with mutations                    | 5      |
| Most REST API GET, HEAD, and OPTIONS requests      | 1      |
| Most REST API POST, PATCH, PUT, or DELETE requests | 5      |

Some REST API endpoints have a different point cost that is not shared publicly.

### [Checking the status of your rate limit](#)

You can use the headers that are sent with each response to determine the current status of your primary rate limit.

| Header name           | Description                                                                                                                                                            |
|-----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| x-ratelimit-limit     | The maximum number of requests that you can make per hour                                                                                                              |
| x-ratelimit-remaining | The number of requests remaining in the current rate limit window                                                                                                      |
| x-ratelimit-used      | The number of requests you have made in the current rate limit window                                                                                                  |
| x-ratelimit-reset     | The time at which the current rate limit window resets, in UTC epoch seconds                                                                                           |
| x-ratelimit-resource  | The rate limit resource that the request counted against. For more information about the different resources, see <a href="#">REST API endpoints for rate limits</a> . |

You can also call the GET `/rate_limit` endpoint to check your rate limit. Calling this endpoint does not count against your primary rate limit, but it can count against your secondary rate limit. See [REST API endpoints for rate limits](#). When possible, you should use the rate limit response headers instead of calling the API to check your rate limit.

There is not a way to check the status of your secondary rate limit.

### [Exceeding the rate limit](#)

If you exceed your primary rate limit, you will receive a 403 or 429 response, and the x-ratelimit-remaining header will be 0. You should not retry your request until after the time specified by the x-ratelimit-reset header.

If you exceed a secondary rate limit, you will receive a 403 or 429 response and an error message that indicates that you exceeded a secondary rate limit. If the `retry-after` response header is present, you should not retry your request until after that many seconds has elapsed. If the `x-ratelimit-remaining` header is 0, you should not retry your request until after the time, in UTC epoch seconds, specified by the `x-ratelimit-reset` header. Otherwise, wait for at least one minute before retrying. If your request continues to fail due to a secondary rate limit, wait for an exponentially increasing amount of time between retries, and throw an error after a specific number of retries.

Continuing to make requests while you are rate limited may result in the banning of your integration.

### **Staying under the rate limit**

You should follow best practices to help you stay under the rate limits. See [Best practices for using the REST API](#).

### **Getting a higher rate limit**

If you want a higher primary rate limit, consider making authenticated requests instead of unauthenticated requests. Authenticated requests have a significantly higher rate limit than unauthenticated requests.

If you are using a personal access token for automation in your organization, consider whether a GitHub App will work instead. The rate limit for GitHub Apps using an installation access token scales with the number of repositories and number of organization users. See [About creating GitHub Apps](#).

If you are using GitHub Apps or OAuth apps, consider upgrading to GitHub Enterprise Cloud. GitHub Apps or OAuth apps have higher rate limits for organizations that use GitHub Enterprise Cloud.

## **Using pagination in the REST API**

Learn how to navigate through paginated responses from the REST API.

### **In this article**

- 

[About pagination](#)

- 

[Using link headers](#)

- 

[Changing the number of items per page](#)

- 

[Scripting with pagination](#)

## [About pagination](#)

When a response from the REST API would include many results, GitHub will paginate the results and return a subset of the results. For example, `GET /repos/octocat/Spoon-Knife/issues` will only return 30 issues from the octocat/Spoon-Knife repository even though the repository includes over 1600 open issues. This makes the response easier to handle for servers and for people.

You can use the link header from the response to request additional pages of data. If an endpoint supports the `per_page` query parameter, you can control how many results are returned on a page.

This article demonstrates how to request additional pages of results for paginated responses, how to change the number of results returned on each page, and how to write a script to fetch multiple pages of results.

## [Using link headers](#)

When a response is paginated, the response headers will include a link header. If the endpoint does not support pagination, or if all results fit on a single page, the link header will be omitted.

The link header contains URLs that you can use to fetch additional pages of results. For example, the previous, next, first, and last page of results.

To see the response headers for a particular endpoint, you can use `curl`, GitHub CLI, or a library you're using to make requests. To see the response headers if you are using a library to make requests, follow the documentation for that library. To see the response headers if you are using `curl` or GitHub CLI, pass the `--include` flag with your request. For example:

```
curl --include --request GET \
--url "https://api.github.com/repos/octocat/Spoon-Knife/issues" \
--header "Accept: application/vnd.github+json"
```

If the response is paginated, the link header will look something like this:

```
link: <https://api.github.com/repositories/1300192/issues?page=2>; rel="prev",
<https://api.github.com/repositories/1300192/issues?page=4>; rel="next",
<https://api.github.com/repositories/1300192/issues?page=515>; rel="last",
<https://api.github.com/repositories/1300192/issues?page=1>; rel="first"
```

The link header provides the URL for the previous, next, first, and last page of results:

- The URL for the previous page is followed by `rel="prev"`.
- The URL for the next page is followed by `rel="next"`.
- The URL for the last page is followed by `rel="last"`.
- The URL for the first page is followed by `rel="first"`.

In some cases, only a subset of these links are available. For example, the link to the previous page won't be included if you are on the first page of results, and the link to the last page won't be included if it can't be calculated.

You can use the URLs from the link header to request another page of results. For example, to request the last page of results based on the previous example:

```
curl --include --request GET \
--url "https://api.github.com/repositories/1300192/issues?page=515" \
--header "Accept: application/vnd.github+json"
```

The URLs in the link header use query parameters to indicate which page of results to return. The query parameters in the link URLs may differ between endpoints, however each paginated endpoint will use the page, before/after, or since query parameters. (Some endpoints use the since parameter for something other than pagination.) In all cases, you can use the URLs in the link header to fetch additional pages of results. For more information about query parameters see [Getting started with the REST API](#).

### [Changing the number of items per page](#)

If an endpoint supports the per\_page query parameter, then you can control how many results are returned on a page. For more information about query parameters see [Getting started with the REST API](#).

For example, this request uses the per\_page query parameter to return two items per page:

```
curl --include --request GET \
--url "https://api.github.com/repos/octocat/Spoon-Knife/issues?per_page=2" \
--header "Accept: application/vnd.github+json"
```

The per\_page parameter will automatically be included in the link header. For example:

```
link: <https://api.github.com/repositories/1300192/issues?per_page=2&page=2>; rel="next",
<https://api.github.com/repositories/1300192/issues?per_page=2&page=7715>; rel="last"
```

### [Scripting with pagination](#)

Instead of manually copying URLs from the link header, you can write a script to fetch multiple pages of results.

The following examples use JavaScript and GitHub's Octokit.js library. For more information about Octokit.js, see [Getting started with the REST API](#) and [the Octokit.js README](#).

### [Example using the Octokit.js pagination method](#)

To fetch paginated results with Octokit.js, you can use octokit.paginate(). octokit.paginate() will fetch the next page of results until it reaches the last page and then return all of the results as a single array. A few endpoints return paginated results as array in an object, as opposed to returning the paginated results as an array. octokit.paginate() always returns an array of items even if the raw result was an object.

For example, this script gets all of the issues from the octocat/Spoon-Knife repository. Although it requests 100 issues at a time, the function won't return until the last page of data is reached.

JavaScript

```
import { Octokit } from "octokit";
```



```
const octokit = new Octokit({ });

const data = await octokit.paginate("GET /repos/{owner}/{repo}/issues", {
  owner: "octocat",
  repo: "Spoon-Knife",
  per_page: 100,
  headers: {
    "X-GitHub-API-Version": "2022-11-28",
  },
});

console.log(data)
```

You can pass an optional map function to `octokit.paginate()` to end pagination before the last page is reached or to reduce memory usage by keeping only a subset of the response. You can also use `octokit.paginate.iterator()` to iterate through a single page at a time instead of requesting every page. For more information, see [the Octokit.js documentation](#).

### Example creating a pagination method

If you are using another language or library that doesn't have a pagination method, you can build your own pagination method. This example still uses the Octokit.js library to make requests, but does not rely on `octokit.paginate()`.

The `getPaginatedData` function makes a request to an endpoint with `octokit.request()`. The data from the response is processed by `parseData`, which handles cases where no data is returned or cases where the data that is returned is an object instead of an array. The processed data is then appended to a list that contains all of the paginated data collected so far. If the response includes a link header and if the link header includes a link for the next page, then the function uses a RegEx pattern (`nextPattern`) to get the URL for the next page. The function then repeats the previous steps, now using this new URL. Once the link header no longer includes a link to the next page, all of the results are returned.

JavaScript

```
import { Octokit } from "octokit";

const octokit = new Octokit({ });

async function getPaginatedData(url) {
  const nextPattern = /(?!<=<)([\\S]*)(?!>=>; rel="Next")/i;
```

```
let pagesRemaining = true;
```

```
let data = [];
```

```
while (pagesRemaining) {
```

```
  const response = await octokit.request(`GET ${url}`, {
```

```
    per_page: 100,
```

```
    headers: {
```

```
      "X-GitHub-API-Version":
```

```
      "2022-11-28",
```

```
    },
```

```
  });
```

```
  const parsedData = parseData(response.data)
```

```
  data = [...data, ...parsedData];
```

```
  const linkHeader = response.headers.link;
```

```
  pagesRemaining = linkHeader && linkHeader.includes(`rel="next"`);
```

```
  if (pagesRemaining) {
```

```
    url = linkHeader.match(nextPattern)[0];
```

```
  }
```

```
}
```

```
return data;
```

```
}
```

```
function parseData(data) {
```

```
  // If the data is an array, return that
```

```
  if (Array.isArray(data)) {
```

```
    return data
```

```

    }

    // Some endpoints respond with 204 No Content instead of empty array
    // when there is no data. In that case, return an empty array.
    if (!data) {
        return []
    }

    // Otherwise, the array of items that we want is in an object
    // Delete keys that don't include the array of items
    delete data.incomplete_results;
    delete data.repository_selection;
    delete data.total_count;
    // Pull out the array of items
    const namespaceKey = Object.keys(data)[0];
    data = data[namespaceKey];

    return data;
}

const data = await getPaginatedData("/repos/octocat/Spoon-Knife/issues");

console.log(data);

```

### Libraries for the REST API

You can use the official Octokit libraries and other third-party libraries to extend and simplify how you use the GitHub API.

### In this article

- 

[About libraries](#)

-

## [Official GitHub libraries](#)

- 

## [Third-party libraries](#)

### [About libraries](#)

You can use libraries to extend and simplify the way your application interacts with GitHub's API. Each library provides pre-built code for a specific programming language. After integrating a library into your project, you can use the pre-built code modules to interact with GitHub's API via a specific programming language.

GitHub maintains official Octokit libraries for some languages. There are also third-party libraries that you can use with GitHub's API, which are not maintained by GitHub.

### [Official GitHub libraries](#)

GitHub maintains these official client libraries for the GitHub API. These repositories are open source, and community contributions are welcome.

For more information, see [Scripting with the REST API and JavaScript](#) and [Scripting with the REST API and Ruby](#).

- JavaScript: [octokit.js](#)
- Ruby: [octokit.rb](#)
- .NET: [octokit.net](#)
- Terraform: [terraform-provider-github](#)

### [Third-party libraries](#)

The following are examples of third-party libraries that you can use to interact with the GitHub API in various programming languages.

These third-party libraries are not maintained by GitHub. Libraries provided by third parties are governed by separate terms of service, privacy policy, and support documentation.

### [Clojure](#)

- Tentacles: [clj-commons/tentacles](#)

### [Dart](#)

- github.dart: [SpinlockLabs/github.dart](#)

### [Emacs Lisp](#)

- gh.el: [sigma/gh.el](#)

### [Go](#)

- go-github: [google/go-github](#)

## Haskell

- haskell-github: [haskell-github/github](https://github.com/haskell-github/github)

## Java

- GitHub API for Java, an object oriented representation of the GitHub API: [hub4j/github-api](https://github.com/hub4j/github-api)
- JCabi GitHub API, based on Java7 JSON API (JSR-353), simplifies tests with a runtime GitHub stub, and covers the entire API: [github.jcabi.com \(Personal Website\)](https://github.com/jcabi/jcabi-github)

## JavaScript

- NodeJS GitHub library: [pksunkara/octonode](https://github.com/pksunkara/octonode)
- Github.js wrapper around the GitHub API: [github-tools/github](https://github.com/github-tools/github)
- Promise-Based CoffeeScript library for the Browser or NodeJS: [philschatz/github-client](https://github.com/philschatz/github-client)

## Julia

- GitHub.jl: [JuliaWeb/GitHub.jl](https://github.com/JuliaWeb/GitHub.jl)

## OCaml

- ocaml-github: [mirage/ocaml-github](https://github.com/mirage/ocaml-github)

## Perl

- Pithub: [plu/Pithub](https://github.com/plu/Pithub)
- Net::GitHub: [fayland/perl-net-github](https://github.com/fayland/perl-net-github)

## PHP

- PHP GitHub API: [KnpLabs/php-github-api](https://github.com/KnpLabs/php-github-api)
- GitHub Joomla! Package: [joomla-framework/github-api](https://github.com/joomla-framework/github-api)
- GitHub bridge for Laravel: [GrahamCampbell/Laravel-GitHub](https://github.com/GrahamCampbell/Laravel-GitHub)

## PowerShell

- PowerShellForGitHub: [microsoft/PowerShellForGitHub](https://github.com/microsoft/PowerShellForGitHub)

## Python

- gidgethub: [gidgethub/gidgethub](https://github.com/gidgethub/gidgethub)
- ghapi: [fastai/ghapi](https://github.com/fastai/ghapi)
- PyGithub: [PyGithub/PyGithub](https://github.com/PyGithub/PyGithub)
- libsaas: [duckboard/libsaas](https://github.com/duckboard/libsaas)
- github3.py: [sigmavirus24/github3.py](https://github.com/sigmavirus24/github3.py)
- agithub: [mozilla/agithub](https://github.com/mozilla/agithub)
- github-flask: [github-flask \(Official Website\)](https://github.com/github-flask/github-flask)

- githubkit: [yanyongyu/githubkit](#)
- octokit.py: [khornberg/octokit.py](#)

### Ruby

- GitHub API Gem: [piotrmurach/github](#)

### Rust

- Octocrab: [XAMPPRocky/octocrab](#)

### Scala

- Github4s: [47deg/github4s](#)

### Shell

- ok.sh: [whiteinge/ok.sh](#)

## **Best practices for using the REST API**

Follow these best practices when using GitHub's API.

### **In this article**

- 

#### [Avoid polling](#)

- 

#### [Make authenticated requests](#)

- 

#### [Avoid concurrent requests](#)

- 

#### [Pause between mutative requests](#)

- 

#### [Handle rate limit errors appropriately](#)

- 

#### [Follow redirects](#)

- 

[Do not manually parse URLs](#)

- 

[Use conditional requests if appropriate](#)

- 

[Do not ignore errors](#)

- 

[Further reading](#)

### [Avoid polling](#)

You should subscribe to webhook events instead of polling the API for data. This will help your integration stay within the API rate limit. For more information, see [Webhooks documentation](#).

### [Make authenticated requests](#)

Authenticated requests have a higher primary rate limit than unauthenticated requests. To avoid exceeding the rate limit, you should make authenticated requests. For more information, see [Rate limits for the REST API](#).

### [Avoid concurrent requests](#)

To avoid exceeding secondary rate limits, you should make requests serially instead of concurrently. To achieve this, you can implement a queue system for requests.

### [Pause between mutative requests](#)

If you are making a large number of POST, PATCH, PUT, or DELETE requests, wait at least one second between each request. This will help you avoid secondary rate limits.

### [Handle rate limit errors appropriately](#)

If you receive a rate limit error, you should stop making requests temporarily according to these guidelines:

- If the retry-after response header is present, you should not retry your request until after that many seconds has elapsed.
- If the x-ratelimit-remaining header is 0, you should not make another request until after the time specified by the x-ratelimit-reset header. The x-ratelimit-reset header is in UTC epoch seconds.

- Otherwise, wait for at least one minute before retrying. If your request continues to fail due to a secondary rate limit, wait for an exponentially increasing amount of time between retries, and throw an error after a specific number of retries.

Continuing to make requests while you are rate limited may result in the banning of your integration.

### [Follow redirects](#)

The GitHub REST API uses HTTP redirection where appropriate. You should assume that any request may result in a redirection. Receiving an HTTP redirection is not an error, and you should follow the redirect.

A 301 status code indicates permanent redirection. You should repeat your request to the URL specified by the location header. Additionally, you should update your code to use this URL for future requests.

A 302 or 307 status code indicates temporary redirection. You should repeat your request to the URL specified by the location header. However, you should not update your code to use this URL for future requests.

Other redirection status codes may be used in accordance with HTTP specifications.

### [Do not manually parse URLs](#)

Many API endpoints return URL values for fields in the response body. You should not try to parse these URLs or to predict the structure of future URLs. This can cause your integration to break if GitHub changes the structure of the URL in the future. Instead, you should look for a field that contains the information that you need. For example, the endpoint to create an issue returns an `html_url` field with a value like `https://github.com/octocat/Hello-World/issues/1347` and a `number` field with a value like `1347`. If you need to know the number of the issue, use the `number` field instead of parsing the `html_url` field.

Similarly, you should not try to manually construct pagination queries. Instead, you should use the link headers to determine what pages of results you can request. For more information, see [Using pagination in the REST API](#).

### [Use conditional requests if appropriate](#)

Most endpoints return an `etag` header, and many endpoints return a `last-modified` header. You can use the values of these headers to make conditional GET requests. If the response has not changed, you will receive a 304 Not Modified response. Making a conditional request does not count against your primary rate limit if a 304 response is returned and the request was made while correctly authorized with an `Authorization` header.

For example, if a previous request returned an `etag` header value of `644b5b0155e6404a9cc4bd9d8b1ae730`, you can use the `if-none-match` header in a future request:

```
curl -H "Authorization: Bearer YOUR-TOKEN" https://api.github.com/meta --include --header 'if-none-match: "644b5b0155e6404a9cc4bd9d8b1ae730"'
```

For example, if a previous request returned a `last-modified` header value of `Wed, 25 Oct 2023 19:17:59 GMT`, you can use the `if-modified-since` header in a future request:



```
curl -H "Authorization: Bearer YOUR-TOKEN" https://api.github.com/repos/github/docs --include --header 'if-modified-since: Wed, 25 Oct 2023 19:17:59 GMT'
```

Conditional requests for unsafe methods, such as POST, PUT, PATCH, and DELETE are not supported unless otherwise noted in the documentation for a specific endpoint.

### Do not ignore errors

You should not ignore repeated 4xx and 5xx error codes. Instead, you should ensure that you are correctly interacting with the API. For example, if an endpoint requests a string and you are passing it a numeric value, you will receive a validation error. Similarly, attempting to access an unauthorized or nonexistent endpoint will result in a 4xx error.

Intentionally ignoring repeated validation errors may result in the suspension of your app for abuse.

### Further reading

- [Best practices for using webhooks](#)
- [Best practices for creating a GitHub App](#)

## **Troubleshooting the REST API**

Learn how to diagnose and resolve common problems for the REST API.

### **In this article**

- 

### Rate limit errors

- 

### 404 Not Found for an existing resource

- 

### Missing results

- 

### Requires authentication when using basic authentication

- 

### Timeouts

-

### [Resource not accessible](#)

- 

### [Problems parsing JSON](#)

- 

### [Body should be a JSON object](#)

- 

### [Invalid request](#)

- 

### [Validation Failed](#)

- 

### [Not a supported version](#)

- 

### [User agent required](#)

- 

### [Other errors](#)

- 

### [Further reading](#)

### [Rate limit errors](#)

GitHub enforces rate limits to ensure that the API stays available for all users. For more information, see [Rate limits for the REST API](#).

If you exceed your primary rate limit, you will receive a 403 Forbidden or 429 Too Many Requests response, and the x-ratelimit-remaining header will be 0. If you exceed a secondary rate

limit, you will receive a 403 Forbidden or 429 Too Many Requests response and an error message that indicates that you exceeded a secondary rate limit.

If you receive a rate limit error, you should stop making requests temporarily according to these guidelines:

- If the `retry-after` response header is present, you should not retry your request until after that many seconds has elapsed.
- If the `x-ratelimit-remaining` header is 0, you should not make another request until after the time specified by the `x-ratelimit-reset` header. The `x-ratelimit-reset` header is in UTC epoch seconds.
- Otherwise, wait for at least one minute before retrying. If your request continues to fail due to a secondary rate limit, wait for an exponentially increasing amount of time between retries, and throw an error after a specific number of retries.

Continuing to make requests while you are rate limited may result in the banning of your integration.

For more information about how to avoid exceeding the rate limits, see [Best practices for using the REST API](#).

#### [404 Not Found for an existing resource](#)

If you make a request to access a private resource and your request isn't properly authenticated, you will receive a 404 Not Found response. GitHub uses a 404 Not Found response instead of a 403 Forbidden response to avoid confirming the existence of private repositories.

If you get a 404 Not Found response when you know that the resource that you are requesting exists, you should check your authentication. For example:

- If you are using a personal access token (classic), you should ensure that:
  - The token has the scopes that are required to use the endpoint. For more information, see [Scopes for OAuth apps](#) and [Managing your personal access tokens](#).
  - The owner of the token has any permissions that are required to use the endpoint. For example, if an endpoint can only be used by organization owners, only users that are owners of the affected organization can use the endpoint.
  - The token has not been expired or revoked. For more information, see [Token expiration and revocation](#).
- If you are using a fine-grained personal access token, you should ensure that:
  - The token has the permissions that are required to use the endpoint. For more information about the required permissions, see the documentation for the endpoint.
  - The resource owner that was specified for the token matches the owner of the resource that the endpoint will affect. For more information, see [Managing your personal access tokens](#).
  - The token has access to any private repositories that the endpoint will affect. For more information, see [Managing your personal access tokens](#).

- The owner of the token has any permissions that are required to use the endpoint. For example, if an endpoint can only be used by organization owners, only users that are owners of the affected organization can use the endpoint.
- The token has not been expired or revoked. For more information, see [Token expiration and revocation](#).
- If you are using a GitHub App installation access token, you should ensure that:
  - The GitHub App has the permissions that are required to use the endpoint. For more information about the required permissions, see the documentation for the endpoint.
  - The endpoint is only affecting resources owned by the account where the GitHub App is installed.
  - The GitHub App has access to any repositories that the endpoint will affect.
  - The token has not been expired or revoked. For more information, see [Token expiration and revocation](#).
- If you are using a GitHub App user access token, you should ensure that:
  - The GitHub App has the permissions that are required to use the endpoint. For more information about the required permissions, see the documentation for the endpoint.
  - The user that authorized the token has any permissions that are required to use the endpoint. For example, if an endpoint can only be used by organization owners, only users that are owners of the affected organization can use the endpoint.
  - The GitHub App has access to any repositories that the endpoint will affect.
  - The user has access to any repositories that the endpoint will affect.
  - The user has approved any updated permissions for your GitHub App. For more information, see [Approving updated permissions for a GitHub App](#).
- If you are using an OAuth app user access token, you should ensure that:
  - The token has the scopes that are required to use the endpoint. For more information, see [Scopes for OAuth apps](#).
  - The user that authorized the token has any permissions that are required to use the endpoint. For example, if an endpoint can only be used by organization owners, only users that are owners of the affected organization can use the endpoint.
  - The organization has not blocked OAuth app access, if you are using an endpoint that will affect resources owned by an organization. App owners cannot see whether their app is blocked, but they can instruct users of the app to check this. For more information, see [About OAuth app access restrictions](#).
  - The token has not been expired or revoked. For more information, see [Token expiration and revocation](#).
- If you are using GITHUB\_TOKEN in a GitHub Actions workflow, you should ensure that:

- The endpoint is only affecting resources owned by the repository where the workflow is running. If you need to access resources outside of that repository, such as resources owned by an organization or resources owned by another repository, you should use a personal access token or an access token for a GitHub App.

For more information about authentication, see [Authenticating to the REST API](#).

You should also check for typos in your URL. For example, adding a trailing slash to the endpoint will result in a 404 Not Found. You can refer to the reference documentation for the endpoint to confirm that you have the correct URL.

Additionally, any path parameters must be URL encoded. For example, any slashes in the parameter value must be replaced with %2F. If you don't properly encode any slashes in the parameter name, the endpoint URL will be misinterpreted.

### [Missing results](#)

Most endpoints that return a list of resources support pagination. For most of these endpoints, only the first 30 resources are returned by default. In order to see all of the resources, you need to paginate through the results. For more information, see [Using pagination in the REST API](#).

If you are using pagination correctly and still do not see all of the results that you expect, you should confirm that the authentication credentials that you used have access to all of the expected resources. For example, if you are using a GitHub App installation access token, if the installation was only granted access to a subset of repositories in an organization, any request for all repositories in that organization will return only the repositories that the app installation can access.

### [Requires authentication when using basic authentication](#)

Basic authentication with your username and password is not supported. Instead, you should use a personal access token or an access token for a GitHub App or OAuth app. For more information, see [Authenticating to the REST API](#).

### [Timeouts](#)

If GitHub takes more than 10 seconds to process an API request, GitHub will terminate the request and you will receive a timeout response and a "Server Error" message.

GitHub reserves the right to change the timeout window to protect the speed and reliability of the API.

You can check the status of the REST API at [githubstatus.com](https://githubstatus.com) to determine whether the timeout is due to a problem with the API. You can also try to simplify your request or try your request later. For example, if you are requesting 100 items on a page, you can try requesting fewer items.

### [Resource not accessible](#)

If you are using a GitHub App or fine-grained personal access token and you receive a "Resource not accessible by integration" or "Resource not accessible by personal access token" error, then your token has insufficient permissions. For more information about the required permissions, see the documentation for the endpoint.

You can use the X-Accepted-GitHub-Permissions header to identify the permissions that are required to access the REST API endpoint.

The value of the X-Accepted-GitHub-Permissions header is a comma separated list of the permissions that are required to use the endpoint. Occasionally, you can choose from multiple permission sets. In these cases, multiple comma-separated lists will be separated by a semicolon.

For example:

- X-Accepted-GitHub-Permissions: contents=read means that your GitHub App or fine-grained personal access token needs read access to the contents permission.
- X-Accepted-GitHub-Permissions: pull\_requests=write,contents=read means that your GitHub App or fine-grained personal access token needs write access to the pull request permission and read access to the contents permission.
- X-Accepted-GitHub-Permissions: pull\_requests=read,contents=read; issues=read,contents=read means that your GitHub App or fine-grained personal access token needs either read access to the pull request permission and read access to the contents permission, or read access to the issues permission and read access to the contents permission.

### [Problems parsing JSON](#)

If you send invalid JSON in the request body, you may receive a 400 Bad Request response and a "Problems parsing JSON" error message. You can use a linter or JSON validator to help you identify errors in your JSON.

### [Body should be a JSON object](#)

If the endpoint expects a JSON object and you do not format your request body as a JSON object, you may receive a 400 Bad Request response and a "Body should be a JSON object" error message.

### [Invalid request](#)

If you omit required parameters or you use the wrong type for a parameter, you may receive a 422 Unprocessable Entity response and an "Invalid request" error message. For example, you will get this error if you specify a parameter value as an array but the endpoint is expecting a string. You can refer to the reference documentation for the endpoint to verify that you are using the correct parameter types and that you are including all of the required parameters.

### [Validation Failed](#)

If your request could not be processed, you may receive a 422 Unprocessable Entity response and a "Validation Failed" error message. The response body will include an errors property, which includes a code property to help you diagnose the problem.

| Code          | Description                                                                                                                     |
|---------------|---------------------------------------------------------------------------------------------------------------------------------|
| missing       | A resource does not exist.                                                                                                      |
| missing_field | A parameter that was required was not specified. Review the documentation for the endpoint to see what parameters are required. |
| invalid       | The formatting of a parameter is invalid. Review the endpoint documentation for more specific information.                      |

| Code           | Description                                                                                                                                       |
|----------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| already_exists | Another resource has the same value as one of your parameters. This can happen in resources that must have some unique key (such as label names). |
| unprocessable  | The parameters that were provided were invalid.                                                                                                   |
| custom         | Refer to the message property to diagnose the error.                                                                                              |

### Not a supported version

You should use the X-GitHub-API-Version header to specify an API version. For example:

```
curl --header "X-GitHub-API-Version:2022-11-28" https://api.github.com/zen
```

If you specify a version that does not exist, you will receive a 400 Bad Request error and a message about the version not being supported.

For more information, see [API Versions](#).

### User agent required

Requests without a valid User-Agent header will be rejected. You should use your username or the name of your application for the User-Agent value.

curl sends a valid User-Agent header by default.

### Other errors

If you observe an error that is not addressed here, you should refer to the error message that the API gives you. Most error messages will provide a clue about what is wrong and a link to relevant documentation.

If you observe unexpected failures, you can use [githubstatus.com](https://githubstatus.com) or the [GitHub status API](#) to check for incidents affecting the API.

### **Timezones and the REST API**

Some REST API endpoints allow you to specify timezone information with your request.

Some requests that create new data, such as creating a new commit, allow you to provide timezone information when specifying or generating timestamps.

Note that these rules apply only to data passed to the API, not to data returned by the API. Timestamps returned by the API are in UTC time, ISO 8601 format.

### Determining the timezone for a request

To determine timezone information for applicable API calls, we apply these rules in order of priority:

1. [Explicitly providing an ISO 8601 timestamp with timezone information](#)
2. [Using the Time-Zone header](#)
3. [Using the last known timezone for the user](#)
4. [Defaulting to UTC without other timezone information](#)

### Explicitly providing an ISO 8601 timestamp with timezone information

For API calls that allow for a timestamp to be specified, we use that exact timestamp. These timestamps look something like 2014-02-27T15:05:06+01:00.

An example of this is the API to manage commits. For more information, see [REST API endpoints for Git commits](#).

### [Using the Time-Zone header](#)

It is possible to supply a Time-Zone header, which defines a timezone according to the [list of names from the Olson database](#).

```
curl -H "Time-Zone: Europe/Amsterdam" -X POST https://api.github.com/repos/github-linguist/linguist/contents/new_file.md
```

This means that we generate a timestamp for the moment your API call is made, in the timezone this header defines.

For example, the API to manage contents generates a git commit for each addition or change, and it uses the current time as the timestamp. For more information, see [REST API endpoints for repository contents](#). The Time-Zone header will determine the timezone used for generating that current timestamp.

### [Using the last known timezone for the user](#)

If no Time-Zone header is specified and you make an authenticated call to the API, we use the last known timezone for the authenticated user. The last known timezone is updated whenever you browse the GitHub website.

### [Defaulting to UTC without other timezone information](#)

If the steps above don't result in any information, we use UTC as the timezone.

### **Using CORS and JSONP to make cross-origin requests**

You can make API requests across domains using cross-origin resource sharing (CORS) and JSONP callbacks.

#### **In this article**

- 

[About cross-origin requests](#)

- 

[Cross-origin resource sharing \(CORS\)](#)

- 

[JSON-P callbacks](#)

[About cross-origin requests](#)



A cross-origin request is a request made to a different domain than the one originating the request. For security reasons, most web browsers block cross-origin requests. However, you can use cross-origin resource sharing (CORS) and JSONP callbacks to make cross-origin requests.

### [Cross-origin resource sharing \(CORS\)](#)

The REST API supports cross-origin resource sharing (CORS) for AJAX requests from any origin. For more information, see the [CORS W3C Recommendation](#) and the [HTML 5 Security Guide](#)

Here's a sample request sent from a browser hitting `http://example.com`:

```
$ curl -I https://api.github.com -H "Origin: http://example.com"
```

```
HTTP/2 302
```

```
Access-Control-Allow-Origin: *
```

```
Access-Control-Expose-Headers: ETag, Link, x-ratelimit-limit, x-ratelimit-remaining, x-ratelimit-reset, X-OAuth-Scopes, X-Accepted-OAuth-Scopes, X-Poll-Interval
```

This is what the CORS preflight request looks like:

```
$ curl -I https://api.github.com -H "Origin: http://example.com" -X OPTIONS
```

```
HTTP/2 204
```

```
Access-Control-Allow-Origin: *
```

```
Access-Control-Allow-Headers: Authorization, Content-Type, If-Match, If-Modified-Since, If-None-Match, If-Unmodified-Since, X-Requested-With
```

```
Access-Control-Allow-Methods: GET, POST, PATCH, PUT, DELETE
```

```
Access-Control-Expose-Headers: ETag, Link, x-ratelimit-limit, x-ratelimit-remaining, x-ratelimit-reset, X-OAuth-Scopes, X-Accepted-OAuth-Scopes, X-Poll-Interval
```

```
Access-Control-Max-Age: 86400
```

### [JSON-P callbacks](#)

You can send a `?callback` parameter to any GET call to have the results wrapped in a JSON function. This is typically used when browsers want to embed GitHub content in web pages and avoid cross-domain problems. The response includes the same data output as the regular API, plus the relevant HTTP Header information.

```
$ curl https://api.github.com?callback=foo
```

```
> /**/foo({
>   "meta": {
>     "status": 200,
>     "x-ratelimit-limit": "5000",
>     "x-ratelimit-remaining": "4966",
```

```
> "x-ratelimit-reset": "1372700873",
> "Link": [ // pagination headers and other links
>   ["https://api.github.com?page=2", {"rel": "next"}]
> ]
> },
> "data": {
>   // the data
> }
> })
```

You can write a JavaScript handler to process the callback. Here's a minimal example you can try:

```
<html>
<head>
<script type="text/javascript">
function foo(response) {
  var meta = response.meta;
  var data = response.data;
  console.log(meta);
  console.log(data);
}

var script = document.createElement('script');
script.src = 'https://api.github.com?callback=foo';

document.getElementsByTagName('head')[0].appendChild(script);
</script>
</head>

<body>
  <p>Open up your browser's console.</p>
</body>
</html>
```

All of the headers have the same string value as the HTTP Headers, except Link. Link headers are pre-parsed for you and come through as an array of [url, options] tuples.

For example, a link that looks like this:

Link: <url1>; rel="next", <url2>; rel="foo"; bar="baz"

will look like this in the Callback output:

```
{
  "Link": [
    [
      "url1",
      {
        "rel": "next"
      }
    ],
    [
      "url2",
      {
        "rel": "foo",
        "bar": "baz"
      }
    ]
  ]
}
```

### Issue event types

For the REST APIs for issue events and timeline events, learn about each event type, the triggering action on GitHub, and each event's unique properties.

### In this article

- 

[Issue event object common properties](#)

- 

[assigned](#)

- 

[automatic base change failed](#)

- 

[automatic base change succeeded](#)

- 

[base ref changed](#)

- 

[closed](#)

- 

[commented](#)

- 

[committed](#)

- 

[connected](#)

- 

[convert to draft](#)

- 

[converted to discussion](#)

- 

[cross-referenced](#)

-

[demilestoned](#)

- 

[deployed](#)

- 

[deployment environment changed](#)

- 

[disconnected](#)

- 

[head\\_ref deleted](#)

- 

[head\\_ref restored](#)

- 

[head\\_ref force\\_pushed](#)

- 

[labeled](#)

- 

[locked](#)

- 

[mentioned](#)

- 

[marked\\_as\\_duplicate](#)

- 

[merged](#)

- 

[milestoned](#)

- 

[pinned](#)

- 

[ready for review](#)

- 

[referenced](#)

- 

[renamed](#)

- 

[reopened](#)

- 

[review dismissed](#)

- 

[review requested](#)

- 

[review request removed](#)

- 

[reviewed](#)

- 

[subscribed](#)

- 

[transferred](#)

- 

[unassigned](#)

- 

[unlabeled](#)

- 

[unlocked](#)

- 

[unmarked\\_as\\_duplicate](#)

- 

[unpinned](#)

- 

[unsubscribed](#)

- 

[user\\_blocked](#)

Issue events are triggered by activity in issues and pull requests and are available in the REST API for [Issue events](#) and [Timeline events](#). Each event type specifies whether the event is available in the REST API for issue events or timeline events.

GitHub's REST API considers every pull request to be an issue, but not every issue is a pull request. For this reason, the Issue Events and Timeline Events endpoints may return both issues and pull requests in the response. Pull requests have a `pull_request` property in the issue object. Because pull requests are issues, issue and pull request numbers do not overlap in a repository. For example, if you open your first issue in a repository, the number will be 1. If you then open a pull request, the number will be 2. Each event type specifies if the event occurs in pull request, issues, or both.

#### [Issue event object common properties](#)

Issue events all have the same object structure, except events that are only available in the REST API for timeline events. Some events also include additional properties that provide more context about the event resources. Refer to the specific event for details about any properties that differ from this object format.

| Name       | Type    | Description                                                        |
|------------|---------|--------------------------------------------------------------------|
| id         | integer | The unique identifier of the event.                                |
| node_id    | string  | The <a href="#">Global Node ID</a> of the event.                   |
| url        | string  | The REST API URL for fetching the event.                           |
| actor      | object  | The person who generated the event.                                |
| event      | string  | Identifies the actual type of event that occurred.                 |
| commit_id  | string  | The SHA of the commit that referenced this issue.                  |
| commit_url | string  | The GitHub REST API link to the commit that referenced this issue. |
| created_at | string  | The timestamp indicating when the event occurred.                  |

#### [assigned](#)

The issue or pull request was assigned to a user.

This event is available for the following issue types.

|               | REST API for issue events | REST API for timeline events |
|---------------|---------------------------|------------------------------|
| Issues        |                           |                              |
| Pull requests |                           |                              |

#### [Properties for assigned](#)

| Name    | Type    | Description                                      |
|---------|---------|--------------------------------------------------|
| id      | integer | The unique identifier of the event.              |
| node_id | string  | The <a href="#">Global Node ID</a> of the event. |



| Name       | Type   | Description                                                                                                                                                |
|------------|--------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| url        | string | The REST API URL for fetching the event.                                                                                                                   |
| actor      | object | The person who generated the event.                                                                                                                        |
| event      | string | Identifies the actual type of event that occurred.                                                                                                         |
| commit_id  | string | The SHA of the commit that referenced this issue.                                                                                                          |
| commit_url | string | The GitHub REST API link to the commit that referenced this issue.                                                                                         |
| created_at | string | The timestamp indicating when the event occurred.                                                                                                          |
| assignee   | object | The person assigned to this issue.                                                                                                                         |
| assigner   | object | The person who performed the assignment for this issue. This field is available in the REST API for issue events but not the REST API for timeline events. |

#### [automatic base change failed](#)

GitHub unsuccessfully attempted to automatically change the base branch of the pull request.

This event is available for the following issue types.

|                      | REST API for issue events | REST API for timeline events |
|----------------------|---------------------------|------------------------------|
| <b>Pull requests</b> |                           |                              |

#### [Properties for automatic base change failed](#)

| Name       | Type    | Description                                                        |
|------------|---------|--------------------------------------------------------------------|
| id         | integer | The unique identifier of the event.                                |
| node_id    | string  | The <a href="#">Global Node ID</a> of the event.                   |
| url        | string  | The REST API URL for fetching the event.                           |
| actor      | object  | The person who generated the event.                                |
| event      | string  | Identifies the actual type of event that occurred.                 |
| commit_id  | string  | The SHA of the commit that referenced this issue.                  |
| commit_url | string  | The GitHub REST API link to the commit that referenced this issue. |
| created_at | string  | The timestamp indicating when the event occurred.                  |

#### [automatic base change succeeded](#)

GitHub successfully attempted to automatically change the base branch of the pull request.

This event is available for the following issue types.

| REST API for issue events | REST API for timeline events |
|---------------------------|------------------------------|
|---------------------------|------------------------------|

Pull requests

[Properties for automatic base change succeeded](#)

| Name       | Type    | Description                                                        |
|------------|---------|--------------------------------------------------------------------|
| id         | integer | The unique identifier of the event.                                |
| node_id    | string  | The <a href="#">Global Node ID</a> of the event.                   |
| url        | string  | The REST API URL for fetching the event.                           |
| actor      | object  | The person who generated the event.                                |
| event      | string  | Identifies the actual type of event that occurred.                 |
| commit_id  | string  | The SHA of the commit that referenced this issue.                  |
| commit_url | string  | The GitHub REST API link to the commit that referenced this issue. |
| created_at | string  | The timestamp indicating when the event occurred.                  |

[base ref changed](#)

The base reference branch of the pull request changed.

This event is available for the following issue types.

| REST API for issue events | REST API for timeline events |
|---------------------------|------------------------------|
|---------------------------|------------------------------|

Pull requests

[Properties for base ref changed](#)

| Name       | Type    | Description                                                        |
|------------|---------|--------------------------------------------------------------------|
| id         | integer | The unique identifier of the event.                                |
| node_id    | string  | The <a href="#">Global Node ID</a> of the event.                   |
| url        | string  | The REST API URL for fetching the event.                           |
| actor      | object  | The person who generated the event.                                |
| event      | string  | Identifies the actual type of event that occurred.                 |
| commit_id  | string  | The SHA of the commit that referenced this issue.                  |
| commit_url | string  | The GitHub REST API link to the commit that referenced this issue. |
| created_at | string  | The timestamp indicating when the event occurred.                  |

[closed](#)

The issue or pull request was closed. When the `commit_id` is present, it identifies the commit that closed the issue using "closes / fixes" syntax. For more information about the syntax, see [Linking a pull request to an issue](#).

This event is available for the following issue types.

|               | REST API for issue events | REST API for timeline events |
|---------------|---------------------------|------------------------------|
| Issues        |                           |                              |
| Pull requests |                           |                              |

#### [Properties for closed](#)

| Name       | Type    | Description                                                        |
|------------|---------|--------------------------------------------------------------------|
| id         | integer | The unique identifier of the event.                                |
| node_id    | string  | The <a href="#">Global Node ID</a> of the event.                   |
| url        | string  | The REST API URL for fetching the event.                           |
| actor      | object  | The person who generated the event.                                |
| event      | string  | Identifies the actual type of event that occurred.                 |
| commit_id  | string  | The SHA of the commit that referenced this issue.                  |
| commit_url | string  | The GitHub REST API link to the commit that referenced this issue. |
| created_at | string  | The timestamp indicating when the event occurred.                  |

#### [commented](#)

A comment was added to the issue or pull request.

This event is available for the following issue types.

|               | REST API for issue events | REST API for timeline events |
|---------------|---------------------------|------------------------------|
| Issues        |                           |                              |
| Pull requests |                           |                              |

#### [Properties for commented](#)

This events properties do not use the same structure as the [issue event object common properties](#). The following properties are returned for this event.

| Name      | Type   | Description                                     |
|-----------|--------|-------------------------------------------------|
| url       | string | The REST API URL to retrieve the issue comment. |
| html_url  | string | The HTML URL of the issue comment.              |
| issue_url | string | The HTML URL of the issue.                      |

| Name               | Type    | Description                                                                                                                                   |
|--------------------|---------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| id                 | integer | The unique identifier of the event.                                                                                                           |
| node_id            | string  | The <a href="#">Global Node ID</a> of the event.                                                                                              |
| user               | object  | The person who commented on the issue.                                                                                                        |
| created_at         | string  | The timestamp indicating when the comment was added.                                                                                          |
| updated_at         | string  | The timestamp indicating when the comment was updated or created, if the comment is never updated.                                            |
| author_association | string  | The permissions the user has in the issue's repository. For example, the value would be "OWNER" if the owner of repository created a comment. |
| body               | string  | The comment body text.                                                                                                                        |
| event              | string  | The event value is "commented".                                                                                                               |
| actor              | object  | The person who generated the event.                                                                                                           |

### [committed](#)

A commit was added to the pull request's HEAD branch.

This event is available for the following issue types.

|                      | REST API for issue events | REST API for timeline events |
|----------------------|---------------------------|------------------------------|
| <b>Pull requests</b> |                           |                              |

### [Properties for committed](#)

This events properties do not use the same structure as the [issue event object common properties](#).

The following properties are returned for this event.

| Name      | Type   | Description                                                  |
|-----------|--------|--------------------------------------------------------------|
| sha       | string | The SHA of the commit in the pull request.                   |
| node_id   | string | The <a href="#">Global Node ID</a> of the event.             |
| url       | string | The REST API URL to retrieve the commit.                     |
| html_url  | string | The HTML URL of the commit.                                  |
| author    | object | The person who authored the commit.                          |
| committer | object | The person who committed the commit on behalf of the author. |
| tree      | object | The Git tree of the commit.                                  |
| message   | string | The commit message.                                          |

| Name                | Type             | Description                                                                                                                   |
|---------------------|------------------|-------------------------------------------------------------------------------------------------------------------------------|
| parents             | array of objects | A list of parent commits.                                                                                                     |
| verification object |                  | The result of verifying the commit's signature. For more information, see <a href="#">REST API endpoint for Git commits</a> . |
| event               | string           | The event value is "committed".                                                                                               |

#### [connected](#)

The issue or pull request was linked to another issue or pull request. For more information, see [Linking a pull request to an issue](#).

This event is available for the following issue types.

|        | REST API for issue events | REST API for timeline events |
|--------|---------------------------|------------------------------|
| Issues |                           |                              |

#### Pull requests

#### [Properties for connected](#)

| Name       | Type    | Description                                                        |
|------------|---------|--------------------------------------------------------------------|
| id         | integer | The unique identifier of the event.                                |
| node_id    | string  | The <a href="#">Global Node ID</a> of the event.                   |
| url        | string  | The REST API URL for fetching the event.                           |
| actor      | object  | The person who generated the event.                                |
| event      | string  | Identifies the actual type of event that occurred.                 |
| commit_id  | string  | The SHA of the commit that referenced this issue.                  |
| commit_url | string  | The GitHub REST API link to the commit that referenced this issue. |
| created_at | string  | The timestamp indicating when the event occurred.                  |

#### [convert to draft](#)

The pull request was converted to draft mode.

This event is available for the following issue types.

|               | REST API for issue events | REST API for timeline events |
|---------------|---------------------------|------------------------------|
| Pull requests |                           |                              |

#### [Properties for convert to draft](#)

| Name       | Type    | Description                                                        |
|------------|---------|--------------------------------------------------------------------|
| id         | integer | The unique identifier of the event.                                |
| node_id    | string  | The <a href="#">Global Node ID</a> of the event.                   |
| url        | string  | The REST API URL for fetching the event.                           |
| actor      | object  | The person who generated the event.                                |
| event      | string  | Identifies the actual type of event that occurred.                 |
| commit_id  | string  | The SHA of the commit that referenced this issue.                  |
| commit_url | string  | The GitHub REST API link to the commit that referenced this issue. |
| created_at | string  | The timestamp indicating when the event occurred.                  |

#### [converted to discussion](#)

The issue was closed and converted to a discussion.

This event is available for the following issue types.

| REST API for issue events | REST API for timeline events |
|---------------------------|------------------------------|
| <b>Issues</b>             |                              |

#### [Properties for converted to discussion](#)

| Name       | Type    | Description                                                        |
|------------|---------|--------------------------------------------------------------------|
| id         | integer | The unique identifier of the event.                                |
| node_id    | string  | The <a href="#">Global Node ID</a> of the event.                   |
| url        | string  | The REST API URL for fetching the event.                           |
| actor      | object  | The person who generated the event.                                |
| event      | string  | Identifies the actual type of event that occurred.                 |
| commit_id  | string  | The SHA of the commit that referenced this issue.                  |
| commit_url | string  | The GitHub REST API link to the commit that referenced this issue. |
| created_at | string  | The timestamp indicating when the event occurred.                  |

#### [cross-referenced](#)

The issue or pull request was referenced from another issue or pull request.

This event is available for the following issue types.

| REST API for issue events | REST API for timeline events |
|---------------------------|------------------------------|
| <b>Issues</b>             |                              |

| REST API for issue events | REST API for timeline events |
|---------------------------|------------------------------|
|---------------------------|------------------------------|

Pull requests

[Properties for cross-referenced](#)

This events properties do not use the same structure as the [issue event object common properties](#). The following properties are returned for this event.

| Name          | Type   | Description                                                                                                                                                                                                                                                                                                                      |
|---------------|--------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| actor         | object | The person who generated the event.                                                                                                                                                                                                                                                                                              |
| created_at    | string | The timestamp indicating when the cross-reference was added.                                                                                                                                                                                                                                                                     |
| updated_at    | string | The timestamp indicating when the cross-reference was updated or created, if the cross-reference never updated.                                                                                                                                                                                                                  |
| source        | object | The issue or pull request that added a cross-reference.                                                                                                                                                                                                                                                                          |
| source[type]  | string | This value will always be "issue" because pull requests are of type issue. Only cross-reference events triggered by issues or pull requests are returned in the REST API for timeline events. To determine the issue that triggered the event is a pull request, you can check if the source[issue][pull_request] object exists. |
| source[issue] | object | The issue object that added the cross-reference.                                                                                                                                                                                                                                                                                 |
| event         | string | The event value is "cross-referenced".                                                                                                                                                                                                                                                                                           |

[demilestoned](#)

The issue or pull request was removed from a milestone.

This event is available for the following issue types.

| REST API for issue events | REST API for timeline events |
|---------------------------|------------------------------|
|---------------------------|------------------------------|

Issues

Pull requests

[Properties for demilestoned](#)

| Name      | Type    | Description                                        |
|-----------|---------|----------------------------------------------------|
| id        | integer | The unique identifier of the event.                |
| node_id   | string  | The <a href="#">Global Node ID</a> of the event.   |
| url       | string  | The REST API URL for fetching the event.           |
| actor     | object  | The person who generated the event.                |
| event     | string  | Identifies the actual type of event that occurred. |
| commit_id | string  | The SHA of the commit that referenced this issue.  |

| Name             | Type   | Description                                                        |
|------------------|--------|--------------------------------------------------------------------|
| commit_url       | string | The GitHub REST API link to the commit that referenced this issue. |
| created_at       | string | The timestamp indicating when the event occurred.                  |
| milestone        | object | The milestone object.                                              |
| milestone[title] | string | The title of the milestone.                                        |

#### [deployed](#)

The pull request was deployed.

This event is available for the following issue types.

| REST API for issue events | REST API for timeline events |
|---------------------------|------------------------------|
| Pull requests             |                              |

#### [Properties for deployed](#)

| Name       | Type    | Description                                                        |
|------------|---------|--------------------------------------------------------------------|
| id         | integer | The unique identifier of the event.                                |
| node_id    | string  | The <a href="#">Global Node ID</a> of the event.                   |
| url        | string  | The REST API URL for fetching the event.                           |
| actor      | object  | The person who generated the event.                                |
| event      | string  | Identifies the actual type of event that occurred.                 |
| commit_id  | string  | The SHA of the commit that referenced this issue.                  |
| commit_url | string  | The GitHub REST API link to the commit that referenced this issue. |
| created_at | string  | The timestamp indicating when the event occurred.                  |

#### [deployment environment changed](#)

The pull request deployment environment was changed.

This event is available for the following issue types.

| REST API for issue events | REST API for timeline events |
|---------------------------|------------------------------|
| Pull requests             |                              |

#### [Properties for deployment environment changed](#)

| Name    | Type    | Description                                      |
|---------|---------|--------------------------------------------------|
| id      | integer | The unique identifier of the event.              |
| node_id | string  | The <a href="#">Global Node ID</a> of the event. |



| Name       | Type   | Description                                                        |
|------------|--------|--------------------------------------------------------------------|
| url        | string | The REST API URL for fetching the event.                           |
| actor      | object | The person who generated the event.                                |
| event      | string | Identifies the actual type of event that occurred.                 |
| commit_id  | string | The SHA of the commit that referenced this issue.                  |
| commit_url | string | The GitHub REST API link to the commit that referenced this issue. |
| created_at | string | The timestamp indicating when the event occurred.                  |

#### [disconnected](#)

The issue or pull request was unlinked from another issue or pull request. For more information, see [Linking a pull request to an issue](#).

This event is available for the following issue types.

|               | REST API for issue events | REST API for timeline events |
|---------------|---------------------------|------------------------------|
| Issues        |                           |                              |
| Pull requests |                           |                              |

#### [Properties for disconnected](#)

| Name       | Type    | Description                                                        |
|------------|---------|--------------------------------------------------------------------|
| id         | integer | The unique identifier of the event.                                |
| node_id    | string  | The <a href="#">Global Node ID</a> of the event.                   |
| url        | string  | The REST API URL for fetching the event.                           |
| actor      | object  | The person who generated the event.                                |
| event      | string  | Identifies the actual type of event that occurred.                 |
| commit_id  | string  | The SHA of the commit that referenced this issue.                  |
| commit_url | string  | The GitHub REST API link to the commit that referenced this issue. |
| created_at | string  | The timestamp indicating when the event occurred.                  |

#### [head\\_ref deleted](#)

The pull request's HEAD branch was deleted.

This event is available for the following issue types.

|               | REST API for issue events | REST API for timeline events |
|---------------|---------------------------|------------------------------|
| Pull requests |                           |                              |

#### [Properties for head\\_ref deleted](#)

| Name       | Type    | Description                                                        |
|------------|---------|--------------------------------------------------------------------|
| id         | integer | The unique identifier of the event.                                |
| node_id    | string  | The <a href="#">Global Node ID</a> of the event.                   |
| url        | string  | The REST API URL for fetching the event.                           |
| actor      | object  | The person who generated the event.                                |
| event      | string  | Identifies the actual type of event that occurred.                 |
| commit_id  | string  | The SHA of the commit that referenced this issue.                  |
| commit_url | string  | The GitHub REST API link to the commit that referenced this issue. |
| created_at | string  | The timestamp indicating when the event occurred.                  |

#### [head\\_ref\\_restored](#)

The pull request's HEAD branch was restored to the last known commit.

This event is available for the following issue types.

| REST API for issue events | REST API for timeline events |
|---------------------------|------------------------------|
| Pull requests             |                              |

#### [head\\_ref\\_force\\_pushed](#)

The pull request's HEAD branch was force pushed.

This event is available for the following issue types.

| REST API for issue events | REST API for timeline events |
|---------------------------|------------------------------|
| Pull requests             |                              |

#### [Properties for head\\_ref\\_force\\_pushed](#)

| Name       | Type    | Description                                                        |
|------------|---------|--------------------------------------------------------------------|
| id         | integer | The unique identifier of the event.                                |
| node_id    | string  | The <a href="#">Global Node ID</a> of the event.                   |
| url        | string  | The REST API URL for fetching the event.                           |
| actor      | object  | The person who generated the event.                                |
| event      | string  | Identifies the actual type of event that occurred.                 |
| commit_id  | string  | The SHA of the commit that referenced this issue.                  |
| commit_url | string  | The GitHub REST API link to the commit that referenced this issue. |
| created_at | string  | The timestamp indicating when the event occurred.                  |

## [labeled](#)

A label was added to the issue or pull request.

This event is available for the following issue types.

|               | REST API for issue events | REST API for timeline events |
|---------------|---------------------------|------------------------------|
| Issues        |                           |                              |
| Pull requests |                           |                              |

### [Properties for labeled](#)

| Name         | Type    | Description                                                        |
|--------------|---------|--------------------------------------------------------------------|
| id           | integer | The unique identifier of the event.                                |
| node_id      | string  | The <a href="#">Global Node ID</a> of the event.                   |
| url          | string  | The REST API URL for fetching the event.                           |
| actor        | object  | The person who generated the event.                                |
| event        | string  | Identifies the actual type of event that occurred.                 |
| commit_id    | string  | The SHA of the commit that referenced this issue.                  |
| commit_url   | string  | The GitHub REST API link to the commit that referenced this issue. |
| created_at   | string  | The timestamp indicating when the event occurred.                  |
| label        | object  | The label added or removed from the issue                          |
| label[name]  | string  | The name of the label.                                             |
| label[color] | string  | The hex color code.                                                |

## [locked](#)

The issue or pull request was locked.

This event is available for the following issue types.

|               | REST API for issue events | REST API for timeline events |
|---------------|---------------------------|------------------------------|
| Issues        |                           |                              |
| Pull requests |                           |                              |

### [Properties for locked](#)

| Name    | Type    | Description                                      |
|---------|---------|--------------------------------------------------|
| id      | integer | The unique identifier of the event.              |
| node_id | string  | The <a href="#">Global Node ID</a> of the event. |

| Name        | Type   | Description                                                                       |
|-------------|--------|-----------------------------------------------------------------------------------|
| url         | string | The REST API URL for fetching the event.                                          |
| actor       | object | The person who generated the event.                                               |
| event       | string | Identifies the actual type of event that occurred.                                |
| commit_id   | string | The SHA of the commit that referenced this issue.                                 |
| commit_url  | string | The GitHub REST API link to the commit that referenced this issue.                |
| created_at  | string | The timestamp indicating when the event occurred.                                 |
| lock_reason | string | The reason an issue or pull request conversation was locked, if one was provided. |

#### [mentioned](#)

The actor was @mentioned in an issue or pull request body.

This event is available for the following issue types.

|        | REST API for issue events | REST API for timeline events |
|--------|---------------------------|------------------------------|
| Issues |                           |                              |

#### Pull requests

#### [Properties for mentioned](#)

| Name       | Type    | Description                                                        |
|------------|---------|--------------------------------------------------------------------|
| id         | integer | The unique identifier of the event.                                |
| node_id    | string  | The <a href="#">Global Node ID</a> of the event.                   |
| url        | string  | The REST API URL for fetching the event.                           |
| actor      | object  | The person who generated the event.                                |
| event      | string  | Identifies the actual type of event that occurred.                 |
| commit_id  | string  | The SHA of the commit that referenced this issue.                  |
| commit_url | string  | The GitHub REST API link to the commit that referenced this issue. |
| created_at | string  | The timestamp indicating when the event occurred.                  |

#### [marked as duplicate](#)

A user with write permissions marked an issue as a duplicate of another issue, or a pull request as a duplicate of another pull request.

This event is available for the following issue types.

|        | REST API for issue events | REST API for timeline events |
|--------|---------------------------|------------------------------|
| Issues |                           |                              |

## Pull requests

### [Properties for marked as duplicate](#)

| Name       | Type    | Description                                                        |
|------------|---------|--------------------------------------------------------------------|
| id         | integer | The unique identifier of the event.                                |
| node_id    | string  | The <a href="#">Global Node ID</a> of the event.                   |
| url        | string  | The REST API URL for fetching the event.                           |
| actor      | object  | The person who generated the event.                                |
| event      | string  | Identifies the actual type of event that occurred.                 |
| commit_id  | string  | The SHA of the commit that referenced this issue.                  |
| commit_url | string  | The GitHub REST API link to the commit that referenced this issue. |
| created_at | string  | The timestamp indicating when the event occurred.                  |

### [merged](#)

The pull request was merged. The commit\_id attribute is the SHA1 of the HEAD commit that was merged. The commit\_repository is always the same as the main repository.

This event is available for the following issue types.

|               | REST API for issue events | REST API for timeline events |
|---------------|---------------------------|------------------------------|
| Pull requests |                           |                              |

### [Properties for merged](#)

| Name       | Type    | Description                                                        |
|------------|---------|--------------------------------------------------------------------|
| id         | integer | The unique identifier of the event.                                |
| node_id    | string  | The <a href="#">Global Node ID</a> of the event.                   |
| url        | string  | The REST API URL for fetching the event.                           |
| actor      | object  | The person who generated the event.                                |
| event      | string  | Identifies the actual type of event that occurred.                 |
| commit_id  | string  | The SHA of the commit that referenced this issue.                  |
| commit_url | string  | The GitHub REST API link to the commit that referenced this issue. |
| created_at | string  | The timestamp indicating when the event occurred.                  |

### [milestoned](#)

The issue or pull request was added to a milestone.

This event is available for the following issue types.

| REST API for issue events | REST API for timeline events |
|---------------------------|------------------------------|
|---------------------------|------------------------------|

Issues

Pull requests

#### [Properties for milestone](#)

| Name             | Type    | Description                                                        |
|------------------|---------|--------------------------------------------------------------------|
| id               | integer | The unique identifier of the event.                                |
| node_id          | string  | The <a href="#">Global Node ID</a> of the event.                   |
| url              | string  | The REST API URL for fetching the event.                           |
| actor            | object  | The person who generated the event.                                |
| event            | string  | Identifies the actual type of event that occurred.                 |
| commit_id        | string  | The SHA of the commit that referenced this issue.                  |
| commit_url       | string  | The GitHub REST API link to the commit that referenced this issue. |
| created_at       | string  | The timestamp indicating when the event occurred.                  |
| milestone        | object  | The milestone object.                                              |
| milestone[title] | string  | The title of the milestone.                                        |

#### [pinned](#)

The issue was pinned.

This event is available for the following issue types.

| REST API for issue events | REST API for timeline events |
|---------------------------|------------------------------|
|---------------------------|------------------------------|

Issues

#### [Properties for pinned](#)

| Name    | Type    | Description                                        |
|---------|---------|----------------------------------------------------|
| id      | integer | The unique identifier of the event.                |
| node_id | string  | The <a href="#">Global Node ID</a> of the event.   |
| url     | string  | The REST API URL for fetching the event.           |
| actor   | object  | The person who generated the event.                |
| event   | string  | Identifies the actual type of event that occurred. |

| Name       | Type   | Description                                                        |
|------------|--------|--------------------------------------------------------------------|
| commit_id  | string | The SHA of the commit that referenced this issue.                  |
| commit_url | string | The GitHub REST API link to the commit that referenced this issue. |
| created_at | string | The timestamp indicating when the event occurred.                  |

#### [ready for review](#)

A draft pull request was marked as ready for review.

This event is available for the following issue types.

|                      | REST API for issue events | REST API for timeline events |
|----------------------|---------------------------|------------------------------|
| <b>Pull requests</b> |                           |                              |

#### [Properties for ready for review](#)

| Name       | Type    | Description                                                        |
|------------|---------|--------------------------------------------------------------------|
| id         | integer | The unique identifier of the event.                                |
| node_id    | string  | The <a href="#">Global Node ID</a> of the event.                   |
| url        | string  | The REST API URL for fetching the event.                           |
| actor      | object  | The person who generated the event.                                |
| event      | string  | Identifies the actual type of event that occurred.                 |
| commit_id  | string  | The SHA of the commit that referenced this issue.                  |
| commit_url | string  | The GitHub REST API link to the commit that referenced this issue. |
| created_at | string  | The timestamp indicating when the event occurred.                  |

#### [referenced](#)

The issue was referenced from a commit message. The commit\_id attribute is the commit SHA1 of where that happened and the commit\_repository is where that commit was pushed.

This event is available for the following issue types.

|               | REST API for issue events | REST API for timeline events |
|---------------|---------------------------|------------------------------|
| <b>Issues</b> |                           |                              |

#### **Pull requests**

#### [Properties for referenced](#)

| Name | Type    | Description                         |
|------|---------|-------------------------------------|
| id   | integer | The unique identifier of the event. |

| Name       | Type   | Description                                                        |
|------------|--------|--------------------------------------------------------------------|
| node_id    | string | The <a href="#">Global Node ID</a> of the event.                   |
| url        | string | The REST API URL for fetching the event.                           |
| actor      | object | The person who generated the event.                                |
| event      | string | Identifies the actual type of event that occurred.                 |
| commit_id  | string | The SHA of the commit that referenced this issue.                  |
| commit_url | string | The GitHub REST API link to the commit that referenced this issue. |
| created_at | string | The timestamp indicating when the event occurred.                  |

### [renamed](#)

The issue or pull request title was changed.

This event is available for the following issue types.

|        | REST API for issue events | REST API for timeline events |
|--------|---------------------------|------------------------------|
| Issues |                           |                              |

### Pull requests

### [Properties for renamed](#)

| Name         | Type    | Description                                                        |
|--------------|---------|--------------------------------------------------------------------|
| id           | integer | The unique identifier of the event.                                |
| node_id      | string  | The <a href="#">Global Node ID</a> of the event.                   |
| url          | string  | The REST API URL for fetching the event.                           |
| actor        | object  | The person who generated the event.                                |
| event        | string  | Identifies the actual type of event that occurred.                 |
| commit_id    | string  | The SHA of the commit that referenced this issue.                  |
| commit_url   | string  | The GitHub REST API link to the commit that referenced this issue. |
| created_at   | string  | The timestamp indicating when the event occurred.                  |
| rename       | object  | The name details.                                                  |
| rename[from] | string  | The previous name.                                                 |
| rename[to]   | string  | The new name.                                                      |

### [reopened](#)

The issue or pull request was reopened.



This event is available for the following issue types.

|        | REST API for issue events | REST API for timeline events |
|--------|---------------------------|------------------------------|
| Issues |                           |                              |

Pull requests

#### [Properties for reopened](#)

| Name       | Type    | Description                                                        |
|------------|---------|--------------------------------------------------------------------|
| id         | integer | The unique identifier of the event.                                |
| node_id    | string  | The <a href="#">Global Node ID</a> of the event.                   |
| url        | string  | The REST API URL for fetching the event.                           |
| actor      | object  | The person who generated the event.                                |
| event      | string  | Identifies the actual type of event that occurred.                 |
| commit_id  | string  | The SHA of the commit that referenced this issue.                  |
| commit_url | string  | The GitHub REST API link to the commit that referenced this issue. |
| created_at | string  | The timestamp indicating when the event occurred.                  |

#### [review dismissed](#)

The pull request review was dismissed.

This event is available for the following issue types.

|               | REST API for issue events | REST API for timeline events |
|---------------|---------------------------|------------------------------|
| Pull requests |                           |                              |

#### [Properties for review dismissed](#)

| Name       | Type    | Description                                                        |
|------------|---------|--------------------------------------------------------------------|
| id         | integer | The unique identifier of the event.                                |
| node_id    | string  | The <a href="#">Global Node ID</a> of the event.                   |
| url        | string  | The REST API URL for fetching the event.                           |
| actor      | object  | The person who generated the event.                                |
| event      | string  | Identifies the actual type of event that occurred.                 |
| commit_id  | string  | The SHA of the commit that referenced this issue.                  |
| commit_url | string  | The GitHub REST API link to the commit that referenced this issue. |
| created_at | string  | The timestamp indicating when the event occurred.                  |

| Name                                  | Type   | Description                                                                                                        |
|---------------------------------------|--------|--------------------------------------------------------------------------------------------------------------------|
| dismissed_review                      | object | The information for the dismissed review.                                                                          |
| dismissed_review[state]               | string | The state the pull request was in when it was dismissed. Can be one of: commented, approved, or changes_requested. |
| dismissed_review[review_id]           | string | The unique identifier of the pull request's review.                                                                |
| dismissed_review[dismissal_message]   | string | The message the user included when dismissing the review.                                                          |
| dismissed_review[dismissal_commit_id] | string | The unique identifier of the commit that dismissed the review if one exists.                                       |

#### [review requested](#)

A pull request review was requested.

This event is available for the following issue types.

| REST API for issue events | REST API for timeline events |
|---------------------------|------------------------------|
| <b>Pull requests</b>      |                              |

#### [Properties for review requested](#)

| Name               | Type    | Description                                                        |
|--------------------|---------|--------------------------------------------------------------------|
| id                 | integer | The unique identifier of the event.                                |
| node_id            | string  | The <a href="#">Global Node ID</a> of the event.                   |
| url                | string  | The REST API URL for fetching the event.                           |
| actor              | object  | The person who generated the event.                                |
| event              | string  | Identifies the actual type of event that occurred.                 |
| commit_id          | string  | The SHA of the commit that referenced this issue.                  |
| commit_url         | string  | The GitHub REST API link to the commit that referenced this issue. |
| created_at         | string  | The timestamp indicating when the event occurred.                  |
| requested_reviewer | object  | The person requested to review the pull request.                   |
| review_requester   | object  | The person who requested a review.                                 |

#### [review request removed](#)

A pull request review request was removed.

This event is available for the following issue types.

| REST API for issue events | REST API for timeline events |
|---------------------------|------------------------------|
| <b>Pull requests</b>      |                              |

### [Properties for review\\_request\\_removed](#)

| Name               | Type    | Description                                                        |
|--------------------|---------|--------------------------------------------------------------------|
| id                 | integer | The unique identifier of the event.                                |
| node_id            | string  | The <a href="#">Global Node ID</a> of the event.                   |
| url                | string  | The REST API URL for fetching the event.                           |
| actor              | object  | The person who generated the event.                                |
| event              | string  | Identifies the actual type of event that occurred.                 |
| commit_id          | string  | The SHA of the commit that referenced this issue.                  |
| commit_url         | string  | The GitHub REST API link to the commit that referenced this issue. |
| created_at         | string  | The timestamp indicating when the event occurred.                  |
| requested_reviewer | object  | The person requested to review the pull request.                   |
| review_requester   | object  | The person who requested a review.                                 |

### [reviewed](#)

The pull request was reviewed.

This event is available for the following issue types.

| REST API for issue events | REST API for timeline events |
|---------------------------|------------------------------|
| Pull requests             |                              |

### [Properties for reviewed](#)

This events properties do not use the same structure as the [issue event object common properties](#). The following properties are returned for this event.

| Name         | Type    | Description                                                                                            |
|--------------|---------|--------------------------------------------------------------------------------------------------------|
| id           | integer | The unique identifier of the event.                                                                    |
| node_id      | string  | The <a href="#">Global Node ID</a> of the event.                                                       |
| user         | object  | The person who commented on the issue.                                                                 |
| body         | string  | The review summary text.                                                                               |
| commit_id    | string  | The SHA of the latest commit in the pull request at the time of the review.                            |
| submitted_at | string  | The timestamp indicating when the review was submitted.                                                |
| state        | string  | The state of the submitted review. Can be one of: commented, changes_requested, approved or dismissed. |
| html_url     | string  | The HTML URL of the review.                                                                            |

| Name               | Type   | Description                                                                                                                                   |
|--------------------|--------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| pull_request_url   | string | The REST API URL to retrieve the pull request.                                                                                                |
| author_association | string | The permissions the user has in the issue's repository. For example, the value would be "OWNER" if the owner of repository created a comment. |
| _links             | object | The html_url and pull_request_url.                                                                                                            |
| event              | string | The event value is "reviewed".                                                                                                                |

### [subscribed](#)

Someone subscribed to receive notifications for an issue or pull request.

This event is available for the following issue types.

|               | REST API for issue events | REST API for timeline events |
|---------------|---------------------------|------------------------------|
| Issues        |                           |                              |
| Pull requests |                           |                              |

### [Properties for subscribed](#)

| Name       | Type    | Description                                                        |
|------------|---------|--------------------------------------------------------------------|
| id         | integer | The unique identifier of the event.                                |
| node_id    | string  | The <a href="#">Global Node ID</a> of the event.                   |
| url        | string  | The REST API URL for fetching the event.                           |
| actor      | object  | The person who generated the event.                                |
| event      | string  | Identifies the actual type of event that occurred.                 |
| commit_id  | string  | The SHA of the commit that referenced this issue.                  |
| commit_url | string  | The GitHub REST API link to the commit that referenced this issue. |
| created_at | string  | The timestamp indicating when the event occurred.                  |

### [transferred](#)

The issue was transferred to another repository.

This event is available for the following issue types.

|        | REST API for issue events | REST API for timeline events |
|--------|---------------------------|------------------------------|
| Issues |                           |                              |

### [Properties for transferred](#)

| Name       | Type    | Description                                                        |
|------------|---------|--------------------------------------------------------------------|
| id         | integer | The unique identifier of the event.                                |
| node_id    | string  | The <a href="#">Global Node ID</a> of the event.                   |
| url        | string  | The REST API URL for fetching the event.                           |
| actor      | object  | The person who generated the event.                                |
| event      | string  | Identifies the actual type of event that occurred.                 |
| commit_id  | string  | The SHA of the commit that referenced this issue.                  |
| commit_url | string  | The GitHub REST API link to the commit that referenced this issue. |
| created_at | string  | The timestamp indicating when the event occurred.                  |

### [unassigned](#)

A user was unassigned from the issue.

This event is available for the following issue types.

|        | REST API for issue events | REST API for timeline events |
|--------|---------------------------|------------------------------|
| Issues |                           |                              |

### Pull requests

### [Properties for unassigned](#)

| Name       | Type    | Description                                                                                                                                                  |
|------------|---------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| id         | integer | The unique identifier of the event.                                                                                                                          |
| node_id    | string  | The <a href="#">Global Node ID</a> of the event.                                                                                                             |
| url        | string  | The REST API URL for fetching the event.                                                                                                                     |
| actor      | object  | The person who generated the event.                                                                                                                          |
| event      | string  | Identifies the actual type of event that occurred.                                                                                                           |
| commit_id  | string  | The SHA of the commit that referenced this issue.                                                                                                            |
| commit_url | string  | The GitHub REST API link to the commit that referenced this issue.                                                                                           |
| created_at | string  | The timestamp indicating when the event occurred.                                                                                                            |
| assignee   | object  | The person unassigned from this issue.                                                                                                                       |
| assigner   | object  | The person who performed the unassignment for this issue. This field is available in the REST API for issue events but not the REST API for timeline events. |

### [unlabeled](#)

A label was removed from the issue.

This event is available for the following issue types.

|        | REST API for issue events | REST API for timeline events |
|--------|---------------------------|------------------------------|
| Issues |                           |                              |

Pull requests

#### [Properties for unlabeled](#)

| Name         | Type    | Description                                                        |
|--------------|---------|--------------------------------------------------------------------|
| id           | integer | The unique identifier of the event.                                |
| node_id      | string  | The <a href="#">Global Node ID</a> of the event.                   |
| url          | string  | The REST API URL for fetching the event.                           |
| actor        | object  | The person who generated the event.                                |
| event        | string  | Identifies the actual type of event that occurred.                 |
| commit_id    | string  | The SHA of the commit that referenced this issue.                  |
| commit_url   | string  | The GitHub REST API link to the commit that referenced this issue. |
| created_at   | string  | The timestamp indicating when the event occurred.                  |
| label        | object  | The label added or removed from the issue                          |
| label[name]  | string  | The name of the label.                                             |
| label[color] | string  | The hex color code.                                                |

#### [unlocked](#)

The issue was unlocked.

This event is available for the following issue types.

|        | REST API for issue events | REST API for timeline events |
|--------|---------------------------|------------------------------|
| Issues |                           |                              |

Pull requests

#### [Properties for unlocked](#)

| Name    | Type    | Description                                      |
|---------|---------|--------------------------------------------------|
| id      | integer | The unique identifier of the event.              |
| node_id | string  | The <a href="#">Global Node ID</a> of the event. |
| url     | string  | The REST API URL for fetching the event.         |
| actor   | object  | The person who generated the event.              |

| Name        | Type   | Description                                                                       |
|-------------|--------|-----------------------------------------------------------------------------------|
| event       | string | Identifies the actual type of event that occurred.                                |
| commit_id   | string | The SHA of the commit that referenced this issue.                                 |
| commit_url  | string | The GitHub REST API link to the commit that referenced this issue.                |
| created_at  | string | The timestamp indicating when the event occurred.                                 |
| lock_reason | string | The reason an issue or pull request conversation was locked, if one was provided. |

#### [unmarked as duplicate](#)

An issue that a user had previously marked as a duplicate of another issue is no longer considered a duplicate, or a pull request that a user had previously marked as a duplicate of another pull request is no longer considered a duplicate.

This event is available for the following issue types.

|               | REST API for issue events | REST API for timeline events |
|---------------|---------------------------|------------------------------|
| Issues        |                           |                              |
| Pull requests |                           |                              |

#### [Properties for unmarked as duplicate](#)

| Name       | Type    | Description                                                        |
|------------|---------|--------------------------------------------------------------------|
| id         | integer | The unique identifier of the event.                                |
| node_id    | string  | The <a href="#">Global Node ID</a> of the event.                   |
| url        | string  | The REST API URL for fetching the event.                           |
| actor      | object  | The person who generated the event.                                |
| event      | string  | Identifies the actual type of event that occurred.                 |
| commit_id  | string  | The SHA of the commit that referenced this issue.                  |
| commit_url | string  | The GitHub REST API link to the commit that referenced this issue. |
| created_at | string  | The timestamp indicating when the event occurred.                  |

#### [unpinned](#)

The issue was unpinned.

This event is available for the following issue types.

|        | REST API for issue events | REST API for timeline events |
|--------|---------------------------|------------------------------|
| Issues |                           |                              |

#### [Properties for unpinned](#)

| Name       | Type    | Description                                                        |
|------------|---------|--------------------------------------------------------------------|
| id         | integer | The unique identifier of the event.                                |
| node_id    | string  | The <a href="#">Global Node ID</a> of the event.                   |
| url        | string  | The REST API URL for fetching the event.                           |
| actor      | object  | The person who generated the event.                                |
| event      | string  | Identifies the actual type of event that occurred.                 |
| commit_id  | string  | The SHA of the commit that referenced this issue.                  |
| commit_url | string  | The GitHub REST API link to the commit that referenced this issue. |
| created_at | string  | The timestamp indicating when the event occurred.                  |

### [unsubscribed](#)

Someone unsubscribed from receiving notifications for an issue or pull request.

This event is available for the following issue types.

|        | REST API for issue events | REST API for timeline events |
|--------|---------------------------|------------------------------|
| Issues |                           |                              |

### Pull requests

### [Properties for unsubscribed](#)

| Name       | Type    | Description                                                        |
|------------|---------|--------------------------------------------------------------------|
| id         | integer | The unique identifier of the event.                                |
| node_id    | string  | The <a href="#">Global Node ID</a> of the event.                   |
| url        | string  | The REST API URL for fetching the event.                           |
| actor      | object  | The person who generated the event.                                |
| event      | string  | Identifies the actual type of event that occurred.                 |
| commit_id  | string  | The SHA of the commit that referenced this issue.                  |
| commit_url | string  | The GitHub REST API link to the commit that referenced this issue. |
| created_at | string  | The timestamp indicating when the event occurred.                  |

### [user blocked](#)

An organization owner blocked a user from the organization. This was done [through one of the blocked user's comments on the issue](#).

This event is available for the following issue types.



|        | REST API for issue events | REST API for timeline events |
|--------|---------------------------|------------------------------|
| Issues |                           |                              |

Pull requests

[Properties for user\\_blocked](#)

| Name       | Type    | Description                                                        |
|------------|---------|--------------------------------------------------------------------|
| id         | integer | The unique identifier of the event.                                |
| node_id    | string  | The <a href="#">Global Node ID</a> of the event.                   |
| url        | string  | The REST API URL for fetching the event.                           |
| actor      | object  | The person who generated the event.                                |
| event      | string  | Identifies the actual type of event that occurred.                 |
| commit_id  | string  | The SHA of the commit that referenced this issue.                  |
| commit_url | string  | The GitHub REST API link to the commit that referenced this issue. |
| created_at | string  | The timestamp indicating when the event occurred.                  |

### GitHub event types

For the GitHub Events API, learn about each event type, the triggering action on GitHub, and each event's unique properties.

### In this article

- 

[Event object common properties](#)

- 

[CommitCommentEvent](#)

- 

[CreateEvent](#)

- 

[DeleteEvent](#)

- 

[ForkEvent](#)

- 

[GollumEvent](#)

- 

[IssueCommentEvent](#)

- 

[IssuesEvent](#)

- 

[MemberEvent](#)

- 

[PublicEvent](#)

- 

[PullRequestEvent](#)

- 

[PullRequestReviewEvent](#)

- 

[PullRequestReviewCommentEvent](#)

- 

[PullRequestReviewThreadEvent](#)

-

## [PushEvent](#)

- 

## [ReleaseEvent](#)

- 

## [SponsorshipEvent](#)

- 

## [WatchEvent](#)

The Events API can return different types of events triggered by activity on GitHub. Each event response contains shared properties, but has a unique payload object determined by its event type. The [Event object common properties](#) describes the properties shared by all events, and each event type describes the payload properties that are unique to the specific event.

### [Event object common properties](#)

The event objects returned from the Events API endpoints have the same structure.

| Event API attribute name | Type    | Description                                                                                    |
|--------------------------|---------|------------------------------------------------------------------------------------------------|
| id                       | integer | Unique identifier for the event.                                                               |
| type                     | string  | The type of event. Events uses PascalCase for the name.                                        |
| actor                    | object  | The user that triggered the event.                                                             |
| actor.id                 | integer | The unique identifier for the actor.                                                           |
| actor.login              | string  | The username of the actor.                                                                     |
| actor.display_login      | string  | The specific display format of the username.                                                   |
| actor.gravatar_id        | string  | The unique identifier of the Gravatar profile for the actor.                                   |
| actor.url                | string  | The REST API URL used to retrieve the user object, which includes additional user information. |
| actor.avatar_url         | string  | The URL of the actor's profile image.                                                          |
| repo                     | object  | The repository object where the event occurred.                                                |
| repo.id                  | integer | The unique identifier of the repository.                                                       |

| Event API attribute name | Type    | Description                                                                                                                                                                                 |
|--------------------------|---------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| repo.name                | string  | The name of the repository, which includes the owner and repository name. For example, octocat/hello-world is the name of the hello-world repository owned by the octocat personal account. |
| repo.url                 | string  | The REST API URL used to retrieve the repository object, which includes additional repository information.                                                                                  |
| payload                  | object  | The event payload object is unique to the event type. See the event type below for the event API payload object.                                                                            |
| public                   | boolean | Whether the event is visible to all users.                                                                                                                                                  |
| created_at               | string  | The date and time when the event was triggered. It is formatted according to ISO 8601.                                                                                                      |
| org                      | object  | The organization that was chosen by the actor to perform action that triggers the event.<br><i>The property appears in the event object only if it is applicable.</i>                       |
| org.id                   | integer | The unique identifier for the organization.                                                                                                                                                 |
| org.login                | string  | The name of the organization.                                                                                                                                                               |
| org.gravatar_id          | string  | The unique identifier of the Gravatar profile for the organization.                                                                                                                         |
| org.url                  | string  | The REST API URL used to retrieve the organization object, which includes additional organization information.                                                                              |
| org.avatar_url           | string  | The URL of the organization's profile image.                                                                                                                                                |

### [Example WatchEvent event object](#)

This example shows the format of the [WatchEvent](#) response when using the [Events API](#).

HTTP/2 200

Link: <<https://api.github.com/resource?page=2>>; rel="next",

<<https://api.github.com/resource?page=5>>; rel="last"

```
[
  {
    "type": "WatchEvent",
    "public": false,
    "payload": {
    },
    "repo": {
      "id": 3,
      "name": "octocat/Hello-World",
```

```

    "url": "https://api.github.com/repos/octocat/Hello-World"
  },
  "actor": {
    "id": 1,
    "login": "octocat",
    "gravatar_id": "",
    "avatar_url": "https://github.com/images/error/octocat_happy.gif",
    "url": "https://api.github.com/users/octocat"
  },
  "org": {
    "id": 1,
    "login": "github",
    "gravatar_id": "",
    "url": "https://api.github.com/orgs/github",
    "avatar_url": "https://github.com/images/error/octocat_happy.gif"
  },
  "created_at": "2011-09-06T17:26:27Z",
  "id": "12345"
}
]

```

### [CommitCommentEvent](#)

A commit comment is created. The type of activity is specified in the action property of the payload object. For more information, see [REST API endpoints for commit comments](#).

The [event object](#) includes properties that are common for all events. Each event object includes a payload property and the value is unique to each event type. The payload object for this event is described below.

#### [Event payload object for CommitCommentEvent](#)

| Key     | Type   | Description                                  |
|---------|--------|----------------------------------------------|
| action  | string | The action performed. Can be created.        |
| comment | object | The <a href="#">commit comment</a> resource. |

### [CreateEvent](#)

A Git branch or tag is created. For more information, see [REST API endpoints for Git database](#).

The [event object](#) includes properties that are common for all events. Each event object includes a payload property and the value is unique to each event type. The payload object for this event is described below.

#### [Event payload object for CreateEvent](#)

| Key           | Type   | Description                                                                                     |
|---------------|--------|-------------------------------------------------------------------------------------------------|
| ref           | string | The <a href="#">git ref</a> resource, or null if ref_type is repository.                        |
| ref_type      | string | The type of Git ref object created in the repository. Can be either branch, tag, or repository. |
| master_branch | string | The name of the repository's default branch (usually main).                                     |
| description   | string | The repository's current description.                                                           |
| pusher_type   | string | Can be either user or a deploy key.                                                             |

#### [DeleteEvent](#)

A Git branch or tag is deleted. For more information, see the [REST API endpoints for Git database](#) REST API.

The [event object](#) includes properties that are common for all events. Each event object includes a payload property and the value is unique to each event type. The payload object for this event is described below.

#### [Event payload object for DeleteEvent](#)

| Key      | Type   | Description                                                                        |
|----------|--------|------------------------------------------------------------------------------------|
| ref      | string | The <a href="#">git ref</a> resource.                                              |
| ref_type | string | The type of Git ref object deleted in the repository. Can be either branch or tag. |

#### [ForkEvent](#)

A user forks a repository. For more information, see [REST API endpoints for repositories](#).

The [event object](#) includes properties that are common for all events. Each event object includes a payload property and the value is unique to each event type. The payload object for this event is described below.

#### [Event payload object for ForkEvent](#)

| Key    | Type   | Description                                      |
|--------|--------|--------------------------------------------------|
| forkee | object | The created <a href="#">repository</a> resource. |

#### [GollumEvent](#)

A wiki page is created or updated. For more information, see [About wikis](#).

The [event object](#) includes properties that are common for all events. Each event object includes a payload property and the value is unique to each event type. The payload object for this event is described below.

### [Event payload object for GollumEvent](#)

| Key                | Type   | Description                                                          |
|--------------------|--------|----------------------------------------------------------------------|
| pages              | array  | The pages that were updated.                                         |
| pages[][page_name] | string | The name of the page.                                                |
| pages[][title]     | string | The current page title.                                              |
| pages[][action]    | string | The action that was performed on the page. Can be created or edited. |
| pages[][sha]       | string | The latest commit SHA of the page.                                   |
| pages[][html_url]  | string | Points to the HTML wiki page.                                        |

### [IssueCommentEvent](#)

Activity related to an issue or pull request comment. The type of activity is specified in the action property of the payload object. For more information, see the [REST API endpoints for issues](#).

The [event object](#) includes properties that are common for all events. Each event object includes a payload property and the value is unique to each event type. The payload object for this event is described below.

### [Event payload object for IssueCommentEvent](#)

| Key                 | Type   | Description                                                                              |
|---------------------|--------|------------------------------------------------------------------------------------------|
| action              | string | The action that was performed on the comment. Can be one of created, edited, or deleted. |
| changes             | object | The changes to the comment if the action was edited.                                     |
| changes[body][from] | string | The previous version of the body if the action was edited.                               |
| issue               | object | The <a href="#">issue</a> the comment belongs to.                                        |
| comment             | object | The <a href="#">comment</a> itself.                                                      |

### [IssuesEvent](#)

Activity related to an issue. The type of activity is specified in the action property of the payload object. For more information, see the [REST API endpoints for issues](#).

The [event object](#) includes properties that are common for all events. Each event object includes a payload property and the value is unique to each event type. The payload object for this event is described below.

### [Event payload object for IssuesEvent](#)

| Key    | Type   | Description                                                                                                                 |
|--------|--------|-----------------------------------------------------------------------------------------------------------------------------|
| action | string | The action that was performed. Can be one of opened, edited, closed, reopened, assigned, unassigned, labeled, or unlabeled. |

| Key                  | Type   | Description                                                      |
|----------------------|--------|------------------------------------------------------------------|
| issue                | object | The <a href="#">issue</a> itself.                                |
| changes              | object | The changes to the issue if the action was edited.               |
| changes[title][from] | string | The previous version of the title if the action was edited.      |
| changes[body][from]  | string | The previous version of the body if the action was edited.       |
| assignee             | object | The optional user who was assigned or unassigned from the issue. |
| label                | object | The optional label that was added or removed from the issue.     |

### [MemberEvent](#)

Activity related to repository collaborators. The type of activity is specified in the action property of the payload object. For more information, see [REST API endpoints for collaborators](#).

The [event object](#) includes properties that are common for all events. Each event object includes a payload property and the value is unique to each event type. The payload object for this event is described below.

#### [Event payload object for MemberEvent](#)

| Key                           | Type   | Description                                                                                            |
|-------------------------------|--------|--------------------------------------------------------------------------------------------------------|
| action                        | string | The action that was performed. Can be added to indicate a user accepted an invitation to a repository. |
| member                        | object | The <a href="#">user</a> that was added.                                                               |
| changes                       | object | The changes to the collaborator permissions if the action was edited.                                  |
| changes[old_permission][from] | string | The previous permissions of the collaborator if the action was edited.                                 |

### [PublicEvent](#)

When a private repository is made public.

#### [Event payload object for PublicEvent](#)

This event returns an empty payload object.

### [PullRequestEvent](#)

Activity related to pull requests. The type of activity is specified in the action property of the payload object. For more information, see [REST API endpoints for pull requests](#).

The [event object](#) includes properties that are common for all events. Each event object includes a payload property and the value is unique to each event type. The payload object for this event is described below.

#### [Event payload object for PullRequestEvent](#)



| Key                  | Type    | Description                                                                                                                                                                                                              |
|----------------------|---------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| action               | string  | The action that was performed. Can be one of opened, edited, closed, reopened, assigned, unassigned, review_requested, review_rejected, review_approved, review_cancelled, review_reopen, review_reopen_and_synchronize. |
| number               | integer | The pull request number.                                                                                                                                                                                                 |
| changes              | object  | The changes to the comment if the action was edited.                                                                                                                                                                     |
| changes[title][from] | string  | The previous version of the title if the action was edited.                                                                                                                                                              |
| changes[body][from]  | string  | The previous version of the body if the action was edited.                                                                                                                                                               |
| pull_request         | object  | The <a href="#">pull request</a> itself.                                                                                                                                                                                 |
| reason               | string  | The reason the pull request was removed from a merge queue if the action was dequeued.                                                                                                                                   |

#### [PullRequestReviewEvent](#)

Activity related to pull request reviews. The type of activity is specified in the action property of the payload object. For more information, see [REST API endpoints for pull requests](#).

The [event object](#) includes properties that are common for all events. Each event object includes a payload property and the value is unique to each event type. The payload object for this event is described below.

#### [Event payload object for PullRequestReviewEvent](#)

| Key          | Type   | Description                                    |
|--------------|--------|------------------------------------------------|
| action       | string | The action that was performed. Can be created. |
| pull_request | object | The pull request the review pertains to.       |
| review       | object | The review that was affected.                  |

#### [PullRequestReviewCommentEvent](#)

Activity related to pull request review comments in the pull request's unified diff. The type of activity is specified in the action property of the payload object. For more information, see [REST API endpoints for pull requests](#).

The [event object](#) includes properties that are common for all events. Each event object includes a payload property and the value is unique to each event type. The payload object for this event is described below.

#### [Event payload object for PullRequestReviewCommentEvent](#)

| Key                 | Type   | Description                                                   |
|---------------------|--------|---------------------------------------------------------------|
| action              | string | The action that was performed on the comment. Can be created. |
| changes             | object | The changes to the comment if the action was edited.          |
| changes[body][from] | string | The previous version of the body if the action was edited.    |

| Key          | Type   | Description                                              |
|--------------|--------|----------------------------------------------------------|
| pull_request | object | The <a href="#">pull request</a> the comment belongs to. |
| comment      | object | The <a href="#">comment</a> itself.                      |

#### [PullRequestReviewThreadEvent](#)

Activity related to a comment thread on a pull request being marked as resolved or unresolved. The type of activity is specified in the action property of the payload object.

The [event object](#) includes properties that are common for all events. Each event object includes a payload property and the value is unique to each event type. The payload object for this event is described below.

#### [Event payload object for PullRequestReviewThreadEvent](#)

| Key    | Type   | Description                                                                                                                                                                                                                                                                  |
|--------|--------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| action | string | The action that was performed. Can be one of: <ul style="list-style-type: none"> <li>resolved - A comment thread on a pull request was marked as resolved.</li> <li>unresolved - A previously resolved comment thread on a pull request was marked as unresolved.</li> </ul> |

pull\_request object The [pull request](#) the thread pertains to.

|        |        |                               |
|--------|--------|-------------------------------|
| thread | object | The thread that was affected. |
|--------|--------|-------------------------------|

#### [PushEvent](#)

One or more commits are pushed to a repository branch or tag.

The [event object](#) includes properties that are common for all events. Each event object includes a payload property and the value is unique to each event type. The payload object for this event is described below.

#### [Event payload object for PushEvent](#)

| Key           | Type    | Description                                                                                                                                                               |
|---------------|---------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| push_id       | integer | Unique identifier for the push.                                                                                                                                           |
| size          | integer | The number of commits in the push.                                                                                                                                        |
| distinct_size | integer | The number of distinct commits in the push.                                                                                                                               |
| ref           | string  | The full <a href="#">git ref</a> that was pushed. Example: refs/heads/main.                                                                                               |
| head          | string  | The SHA of the most recent commit on ref after the push.                                                                                                                  |
| before        | string  | The SHA of the most recent commit on ref before the push.                                                                                                                 |
| commits       | array   | An array of commit objects describing the pushed commits. (The array includes a maximum of 20 commits. If necessary, you can use the <a href="#">Commits API</a> to fetch |

| Key                      | Type    | Description                                                                                                 |
|--------------------------|---------|-------------------------------------------------------------------------------------------------------------|
|                          |         | additional commits. This limit is applied to timeline events only and isn't applied to webhook deliveries.) |
| commits[][sha]           | string  | The SHA of the commit.                                                                                      |
| commits[][message]       | string  | The commit message.                                                                                         |
| commits[][author]        | object  | The git author of the commit.                                                                               |
| commits[][author][name]  | string  | The git author's name.                                                                                      |
| commits[][author][email] | string  | The git author's email address.                                                                             |
| commits[][url]           | url     | URL that points to the commit API resource.                                                                 |
| commits[][distinct]      | boolean | Whether this commit is distinct from any that have been pushed before.                                      |

### [ReleaseEvent](#)

Activity related to a release. The type of activity is specified in the action property of the payload object. For more information, see the [REST API endpoints for releases and release assets](#) REST API.

The [event object](#) includes properties that are common for all events. Each event object includes a payload property and the value is unique to each event type. The payload object for this event is described below.

#### [Event payload object for ReleaseEvent](#)

| Key                 | Type   | Description                                                |
|---------------------|--------|------------------------------------------------------------|
| action              | string | The action that was performed. Can be published.           |
| changes[body][from] | string | The previous version of the body if the action was edited. |
| changes[name][from] | string | The previous version of the name if the action was edited. |
| release             | object | The <a href="#">release</a> object.                        |

### [SponsorshipEvent](#)

Activity related to a sponsorship listing. The type of activity is specified in the action property of the payload object. For more information, see [About GitHub Sponsors](#).

#### [Event payload object for SponsorshipEvent](#)

| Key            | Type   | Description                                                                                                                          |
|----------------|--------|--------------------------------------------------------------------------------------------------------------------------------------|
| action         | string | The action that was performed. This can be created.                                                                                  |
| effective_date | string | The pending_cancellation and pending_tier_change event types will include the date the cancellation or tier change will take effect. |

| Key                          | Type   | Description                                                                                                                                                                              |
|------------------------------|--------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| changes[tier][from]          | object | The tier_changed and pending_tier_change will include the original tier before the change or pending change. For more information, see the <a href="#">pending tier change payload</a> . |
| changes[privacy_level][from] | string | The edited event types include the details about the change when someone edits sponsorship to change the privacy.                                                                        |

### [WatchEvent](#)

When someone stars a repository. The type of activity is specified in the action property of the payload object. For more information, see [REST API endpoints for activity](#).

The [event object](#) includes properties that are common for all events. Each event object includes a payload property and the value is unique to each event type. The payload object for this event is described below.

### [Event payload object for WatchEvent](#)

| Key    | Type   | Description                                                    |
|--------|--------|----------------------------------------------------------------|
| action | string | The action that was performed. Currently, can only be started. |